



中国科学技术大学

指针的高级应用

吴锋

Email: wufeng02@ustc.edu.cn



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



变量名到地址的映射

- ❖ 简单说：程序运行过程中，CPU根据变量名与内存地址的映射关系，在该变量所在的地址进行数据操作
- ❖ 实际上，从变量名到地址的映射
 - ☞ 程序编译后，变量名映射为逻辑地址
 - ☞ 程序加载到内存中时，逻辑地址映射为物理内存地址
 - ☞ 也有操作系统将逻辑地址映射为逻辑内存地址，再由操作系统维护逻辑内存地址到物理内存地址的映射
- ❖ 程序运行过程中，CPU通过地址对内存单元进行数据操作



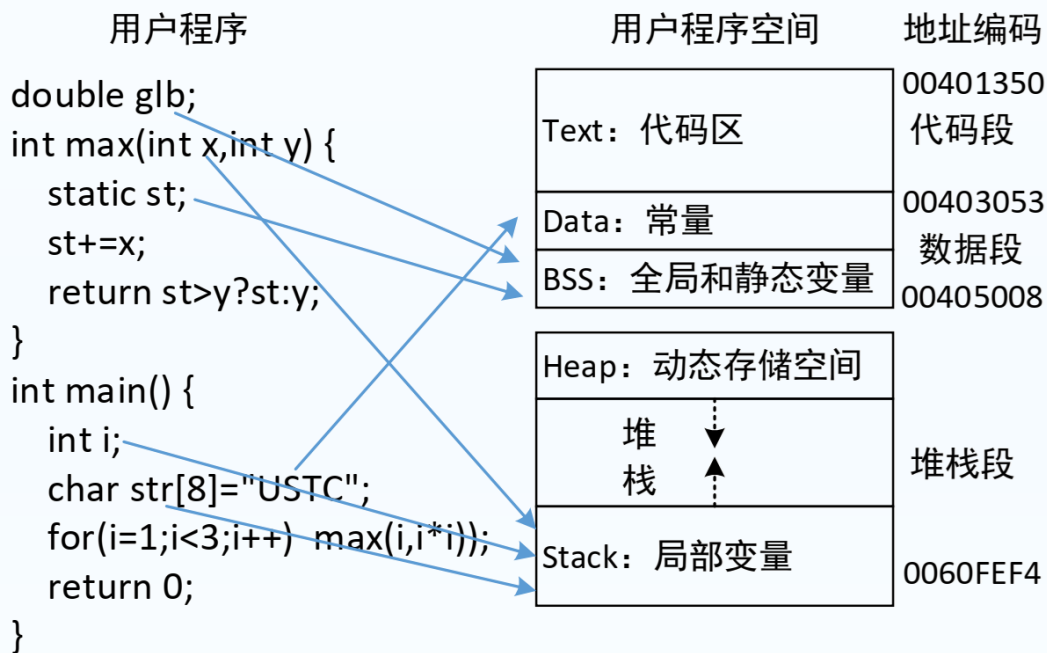
程序空间与数据存储

❖ 程序空间

☞ 操作系统为程序分配内存空间用于加载指令和数据

❖ 程序空间的分段

☞ 代码段、堆栈段、数据段





指针变量的声明

❖ 指向复杂数据类型的指针

- ❧ `int (*ptr)[5];` //ptr是指向由5个整型元素构成的数组的指针变量
- ❧ `int *parr[5];` //parr是一个5个整形指针元素构成的指针数组
- ❧ `int **ip;` //ip是一个“指向整型量的指针变量”的指针变量
- ❧ `int *fip();` //fip是返回整型指针的函数（函数声明）
- ❧ `int (*pti)();` //pti是指向一个返回整数的函数的指针变量
- ❧ `int *(*pfpi)();` //pfpi是一个指向函数的指针变量，该函数返回整数指针

❖ 指针变量也占据内存空间（在32/64位系统上分别占4/8字节）

❖ 调试指针时可使用 `%p` 打印指针所指向的地址

```
int num = 10;
int *p = &num;
printf("Address of num: %p\n", (void *)p);
// 输出地址如: 0x7fffa1c9818c
```



指针相关运算符

- ❖ 取地址运算符`&`，例：`scanf("%d", &a);`
- ❖ 间接访问运算符`*`，例：`*p=*p+10;`
- ❖ 赋值运算符`=`，例：`int a,*p; p=&a;`
- ❖ 与整数加、减运算 `+`, `-`
 - ☞ 例：`int a[10], *p=a; p+n`指向`a[i+n]`，`p-n`指向`a[i-n]`
 - ☞ 例：若指针`q`指向数组`a`的第`j`个元素`a[j]`，则 `p-q`的值即为`i-j`
- ❖ 自增、自减运算`++`, `--`
 - ☞ 例：`p++`的值为`a[i]`的地址，`p`自增后指向`a[i+1]`
 - ☞ 例：`p--`的值为`a[i]`的地址，`p`自减后指向`a[i-1]`
 - ☞ 类似的，可以得到 `++p`, `--p` 的值和指向



指针相关运算符（续）

❖ 关系运算 $>$, $<$, $==$, $>=$, $<=$, $!=$

- ❧ 同类型指针之间可比较是否 $==$ 或 $!=$ ，判断二者是否指向同一个数据
- ❧ 指向同一个数组中的元素的指针可以进行 $>$ 、 $>=$ 、 $<$ 、 $<=$ 的关系比较，判断这两个指针所指元素在数组中的前后次序关系
- ❧ 任何类型的指针都可以与 0 或 $NULL$ 比较是否 $==$ 或 $!=$ ，判断指针值是否为“空”，既是否为空指针

❖ 下标运算 $[]$

- ❧ 等价性： $a[i] \Leftrightarrow *(a+i)$, $*(p+i) \Leftrightarrow p[i]$
- ❧ 若指针 p 指向数组 a 的起始元素 $a[0]$ ， $a[i]$ 就可以表示为 $*(p+i)$
- ❧ 由于 p 和 a 的值相等， $p[i]$ 和 $*(a+i)$ 也可以表示 $a[i]$



特殊指针

- ❖ **野指针**是一个不应存在的**无效指针**，如指针变量未赋值或指针运算超出范围（数组越限），对该指针操作的**后果不确定**
- ❖ **空指针NULL**是C指针类型中的一个特殊值，表示指针变量的值为空，即**不指向任何内存单元**，常用于**初始化**指针变量

如：`int *p=NULL;` `//NULL的值为0，但不是地址0H`

- ❖ **空类型指针**是用**void**声明的一种**通用指针**变量（万能指针）

声明形式：`void *变量名;`

- ∞ 其它任意类型的指针都可以直接赋值给void指针：`void *p = &n;`
- ∞ void指针赋值给其它类型指针时，必须进行强制类型转换
- ∞ void指针不能直接取值或运算，一般应将void指针强制转换成特定类型的指针后再操作：`int *p_int = (int *)p;` `// 转换回int*`



指针的安全使用原则

- ❖ 指针必须初始化: `int *p = NULL;`
- ❖ 使用前检查有效性: `if (p != NULL) *p = 10;`
- ❖ 释放后置为 `NULL`: `free(p); p = NULL;`
- ❖ 避免类型不匹配: `int *p_int = p_char; // 尽量避免`
- ❖ 谨慎使用类型转换: `int *p_int = (int *)p_void; // 谨慎使用`
- ❖ 擅用 `const` 修饰符修饰指针:

```
// 指针指向的内容不可修改
const int *p = NULL; int num = 10;

p = &num; // 允许修改指针所指向的变量
// *p = 20; // 错误: 不可修改指向的值
```

指向常量的指针

```
// 指针本身不可修改
int *const p = &num; // 初始化指针

*p = 20; // 允许修改指针指向变量的值
// p = &other; // 错误: 不可修改指针指向
```

指针常量



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



指向数组的指针

- ❖ 声明变量时的数组名是一个**符号地址常量**，可以理解为指针常量，指向数组的**首元素地址**，是**右值**，不能对数组名赋值
- ❖ 除了数组名的值不能改变外，数组名和指针在很多地方可以通用，例如 `int a[5], *iptr = a;`
 - ⌘ `iptr = a;` 等价于 `iptr = &a[0];` 即取元素地址
 - ⌘ `a[3] ⇔ *(a+3) ⇔ *(iptr+3) ⇔ iptr[3]`
 - ⌘ `*(iptr+3)` 的运算过程：取出指针变量 `iptr` 的值，加上 `3×sizeof(int)`，得到新地址 `&a[3]`，再取值得到内容 `a[3]`
 - ⌘ `a[3]` 的运算过程：取出数组首地址，加上 `3×sizeof(int)`，得到新地址 `&a[3]`，再取值得到内容 `a[3]`



指向数组的指针（例一）

❖ 例：指针与数组的效率比较

```
#include<stdio.h>
void main() {
    int a[5] = {1, 2, 3, 4, 5};
    int i, *p;
    for (i=0;i<5;i++) printf("%6d", a[i]); //下标法
    printf("\n");
    for (i=0;i<5;i++) printf("%6d", *(a+i)); //数组名法
    putchar('\n');
    for (p=a;p<a+5;p++) printf("%6d", *p); //指针变量法
    printf("\n");
} // 3条printf运行结果一样
```

效率相同

效率略高

- ☑ 若是按递增或递减顺序访问数组，用指针，效率高
- ☑ 若不确定多维数组的高维大小，用指针，灵活
- ☑ 若随机访问数组，用下标，简洁明了



指向数组的指针（例二）

❖ 例：猜意图，找错误

```
#include<stdio.h>
void main() {
    int i, a[10], *p;
    p=a;
    for(i=0; i<6; i++) scanf("%d", p++);
    ... ..
    for(i=0; i<6; i++) printf("%6d\t", *p++);
    printf("\n");
}
```

☑ 指针未复位

☑ 数组越界



指向数组的指针（例三）

❖ 例：利用一维指针访问二维数据，并输出

```
#include <stdio.h>

void main() {
    int a[3][4], i, j, *p, n = 0;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 4; j++)
            (*(a+i)+j)=i*4+j;
    for (p = &a[0][0]; p <= &a[2][3]; p++) {
        printf("%6d", *p);
        if (++n%4 == 0) printf("\n");
    }
}
```

程序运行结果

0	1	2	3
4	5	6	7
8	9	10	11



指向数组的指针（例四）

❖ 例：要求不使用下标运算，仅通过指针移动来实现字符串的原地逆序

```
void reverse_string(char *str) {  
    // 1. 安全检查：如果字符串为空或长度为0，直接返回  
    if (str == NULL || *str == '\0') { return; }  
    char *head = str, *tail = str;    // 指向字符串首字符  
    // 2. 将 tail 移动到字符串的最后一个有效字符上（'\0'的前一个）  
    while (*(tail + 1) != '\0') { tail++; }  
    // 3. 交换 head 和 tail 指向的内容，直到它们相遇  
    while (head < tail) {  
        // 经典的交换逻辑  
        char temp = *head; *head = *tail; *tail = temp;  
        // 4. 移动指针  
        head++; // 头指针向后移  
        tail--; // 尾指针向前移  
    }  
}
```

- ☑ 效率：避免每次循环都去计算 `str[i]`（即基地址 + 偏移量），直接自增指针更快
- ☑ 通用性：这种“头尾指针”的思想是所有双指针算法的基础



指向二维数组的指针

❖ 定义形式：类型名 (*变量名)[数组大小]

☞ 如：int (*pa)[4]，pa 是指向 “由4个整型量构成的数组” 的指针变量

❖ 若有 int a[2][4]，则可以 pa = a;

☞ 如：char (*next)[16]，next 是指向 “由16个字符构成的数组” 的指针变量

❖ 若有 char n[5][16]，则可以 next = n;



指向二维数组的指针（例）

- ❖ 例：被调函数的形参分别用指向基本类型的指针变量和指向数组的指针变量实现二维数组传递

```
void print_1(int *p, int n) {  
    ... *(p+i) ...  
}  
  
void print_2(int (*p)[4], int m, int n) {  
    ... (*(p+i)+j) ...  
}  
  
void main() {  
    int a[3][4];  
    int (*pa)[4];  
    ... pa=a; print_1(*pa++, 4) ...  
    ... print_2(a, 3, 4) ...  
}
```



指向二维数组的指针

❖ 数组取值时，每处理一个[]或 *，相当于一次指针取值运算

☞ 如 `int a[3][4];`

☞ `a`、`a+i`，指向“由4个整型数构成的数组”的指针

☞ `a[i]`、`a[i]+j`，指向整型数的指针，即 `*(a+i)`、`*(a+i)+j`

☞ `a[i][j]`，整型数，即 `*(*(a+i)+j)` 或 `*(a+i*4+j)`

❖ 数组取地址时，&表示指向该类型元素的指针

☞ 如 `int a[3][4];`

☞ `&a[i][j]`，指向整型数的指针，值为 `*(a+i)+j` 或 `a[i]+j`

☞ `&a[i]`，指向“由4个整型数构成的数组”的指针，值为 `a+i`

☞ `&a`，指向二维数组的指针，值为 `a`

❖ 注意 `&a` 与 `&a[0]` 的差异

❖ ~~`a[0] = a + 0`~~ `a[0] = *(a + 0)`，二者值相等且类型一样

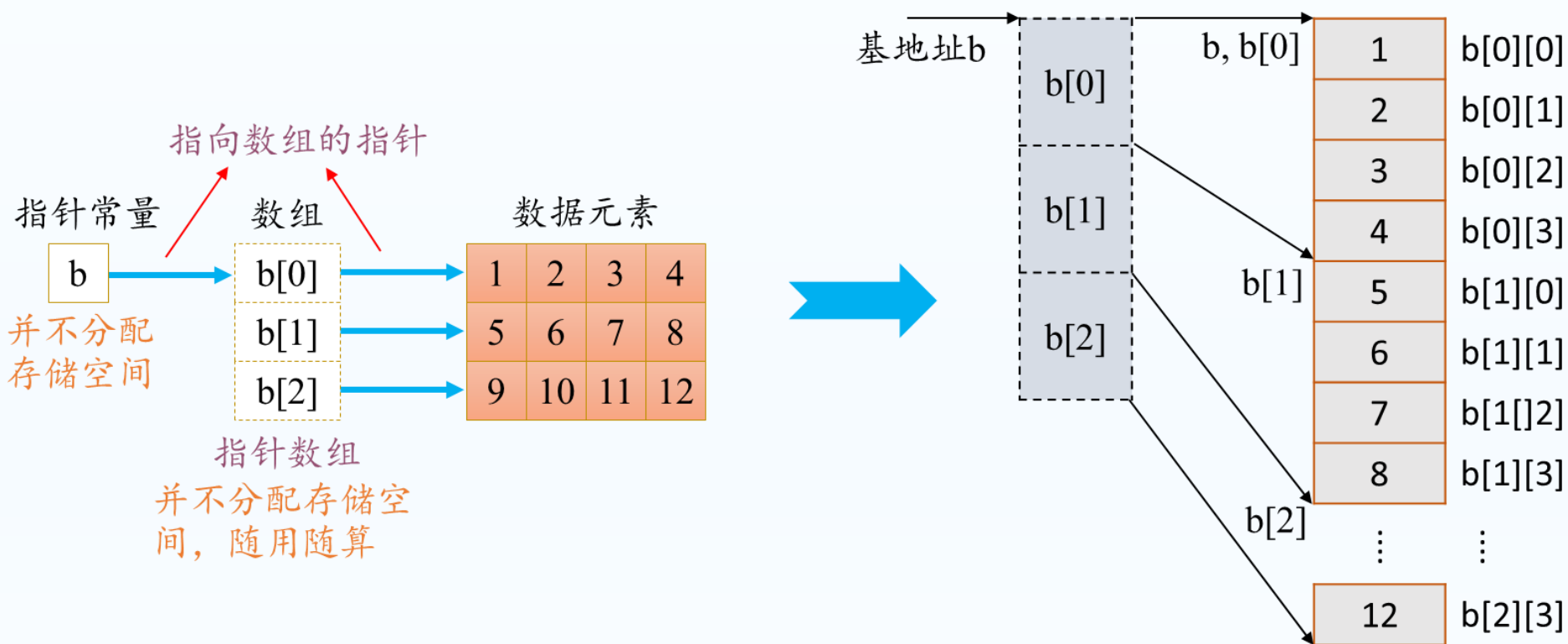
❖ 思考：可以有 `&(a+0)` 吗？

```
printf("%p", &(a+0));  
// error: lvalue required as  
unary '&' operand
```



指向多维数组的指针

- ❖ 多维数组虽然也是连续存储在内存中的元素集合，但语法上（编程使用时）是元素为数组的数组
- ❖ 例如：`int b[3][4]={{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};`





指向多维数组的指针（例）

- ❖ 例：多维数组的指针进行赋值时，应特别注意类型的匹配
 - ☞ `pa` 将以自己的类型（指向包含两个整型数的数组的指针）去理解 `b` 和 `c` 数组中的数据

```
int a[2][2] = {{1,2},{3,4}};  
int b[3][3] = {2,3,4,5,6,7,8,9,10};  
int c[4] = {12,13,14,15}  
int (*pa)[2];  
  
pa = a;  
pa = b; //警告但通过编译  
pa = c; //警告但通过编译
```



指向字符串常量的指针

- ❖ 可以定义一个字符指针变量，让其指向一个字符串常量
 - ❧ 例如：`const char *msg = "Hello";` // 推荐写法
 - `msg[0] = 'h';` // 错误：试图修改字符串常量
 - ❧ 实际上就是把字符串常量在内存中的首地址赋给该指针
 - ❧ 其地位等价于指向一维字符数组第一个元素的指针
 - ❧ 注意这个指针指向常量区，指针值可以改，指针指向的内容不可以改
- ❖ 字符数组与字符串常量的差别
 - ❧ 字符数组存放在程序活动的栈中，可以修改
 - ❧ 字符串常数存放在常量区，不可以修改



指向字符串常量的指针

- ❖ 当字符指针变量指向一个字符串（不需要是首字符）后，可以用数组下标的形式来引用该字符串中的字符

```
const char *pstr = "Hello, world!";  
char str[40];  
int i;  
for(i = 0; *(pstr+i) != '\0'; i++) {  
    str[i] = pstr[i];  
}  
str[i] = '\0'; //此句不能少  
//注意, *(pstr+i)等价于pstr[i], 即访问字符串中第i个字符
```



指向字符串常量的指针

- ❖ 可以用字符串常量对字符指针变量和字符数组初始化
- ❖ 可以用字符串常量对字符指针变量赋值
- ❖ 不可以用字符串常量对数组名赋值（试图修改数组名常量）

```
char *pstr1 = "Hello", str1[40] = "Okay";
```

```
char *pstr2, str2[40];
```

```
pstr2 = "world";
```

```
str2 = "world"; //错误！
```

- ❖ 只能逐个元素地对数组赋值（比如for循环等方法）



指针数组

- ❖ 可以定义由指针作为元素的数组，具体的声明形式：

类型说明符 *数组名[数组长度]

例： `int *p[3]; char *name[]=...`;

`// 注意 int (*p)[3] 与 int *p[3] 的区别`

- ❖ 指针数组中的元素都是指向同一类型的指针，常见的应用：
 - ⌘ 若有大量数据（例如学生信息）需要按某个元素排序时（例如按姓名拼音顺序），使用指针数组排序，就可以避免大量数据的搬移
 - ⌘ 当需要对多个已存在的对象进行批量处理时，也可以使用指针数据
 - ❖ 例：游戏程序中各自独立行动的人物（不使用多线程），每人一个指针组成数组，就可以在一个循环中顺序处理所有人物
 - ❖ 例：多线程程序使用一个指针数组/链表来顺序处理各个进程



指针数组

- ❖ 指针数组常用于处理一组字符串，比使用二维数组更方便，也节省内存空间，在排序等操作中效率更高
 - ∞ 例： `char *name[]={“allen”, “word”, “jones”, “martin”};`
 - ∞ 字符串常量 “allen” 等又称为字符串字面量，它们被存储于常量区
 - ∞ 在代码中，字符串字面量可以等同于一个 `char *` 指针
 - ❖ 甚至允许 “allen”[3] = ‘e’ （虽然编译可以通过，但是执行会发生错误，即常量区不允许改写）
 - ∞ 在这里，相当于在初始化器中列举了一系列字符指针，它们分别指向常数区的字符串



指针数组（例）

❖ 例：使用指针数组存储指向字符串的指针的优点

- ∞ 允许不等长的字符串以相对规整的方式组织在一起，注意它们不一定存储在一起，便于调整各字符串在数组内的位置，修改指针即可

```
#include<string.h>
void main() {
    int n = 4;
    char *name[] = { "allen", "word", "jones", "martin" };
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++)
            if (strcmp(name[i], name[j]) > 0) {
                char *t = name[i];
                name[i] = name[j];
                name[j] = t;
            }
    }
}
```



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



二级指针

- ❖ 二级指针也称为：指向指针变量的指针变量
- ❖ 数据类型：必须与一级指针指向的数据类型一致
- ❖ 初始化要求：必须指向已存在的一级指针的地址
- ❖ 定义的一般形式：
类型名 **指针变量名;

```
int num = 10;  
int *p1 = &num; // 一级指针p1指向num  
int **p2 = &p1; // 二级指针p2指向一级指针p1  
// *p2: 第一次获取p2指向的一级指针p1的值（即num的地址）  
// **p2: 第二次通过p1的值（地址）获取num的实际值10  
printf("%d\n", **p2); // 输出10
```



二级指针（例一）

❖ 例：动态创建一个3行4列的二维数组（矩阵）

```
// 创建3行4列的二维数组（矩阵）
int rows = 3, cols = 4;
// 分配行指针数组
int **matrix = malloc(rows * sizeof(int *));
for (int i = 0; i < rows; i++) {
    // 分配每行的列内存
    matrix[i] = malloc(cols * sizeof(int));
}

// 初始化数组
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = i * cols + j + 1;
    }
}
```



二级指针（例一续）

❖ 例：动态创建一个3行4列的二维数组（矩阵）

```
// 访问元素
printf("%d\n", matrix[1][2]); // 输出7
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%d\n", matrix[i][j]); // 遍历元素
    }
}

// 释放内存
for (int i = 0; i < rows; i++) {
    free(matrix[i]); // 先释放列内存
}
free(matrix); // 再释放行内存
matrix = NULL; // 置为空指针
```



二级指针（例二）

- ❖ 例： 5个学生，每人所修课程门数不同（成绩存放在数组中，以-1表示结束），编写程序输出他们的各项成绩
 - ∞ 每人的成绩各用一个一维数组保存
 - ∞ 指针数组 `int *gread[]`，每个元素指向一个学生的数组
 - ∞ 指向指针的指针 `int **p=gread`，用于指向gread的元素

```
int stu1[]={...}, stu2[]={...}, stu3[]={...},  
    stu4[]={...}, stu5[]={...};  
int *gread[]={stu1, stu2, stu3, stu4, stu5};  
int **p=gread, i;
```




二级指针（例二续）

❖ 例： 5个学生，课程数不同，编写程序输出他们的各项成绩

∞ **p 取出一个成绩； (*p)++ 修改了 gread 数组中某一个元素的值，以指向该学生的下一项成绩； p++ 使得p指向gread中的下一个元素

```
for (i=1; i<=5; i++) {  
    printf("student %d gread:", i);  
    while (**p>0) {  
        printf("%4d", **p);  
        (*p)++;  
    }  
    p++;  
    printf("\n");  
}
```

```
int stu1[]={...}, stu2[]={...}, stu3[]={...},  
    stu4[]={...}, stu5[]={...};  
int *gread[]={stu1, stu2, stu3, stu4, stu5};  
int **p=gread, i;
```



二级指针的安全使用原则

- ❖ 初始化规则：二级指针必须指向有效的一级指针地址

☞ 例：`int **p2; **p2 = 10;`

`// 错误：p2未初始化，指向不确定地址`

- ❖ 双重检查：解引用前检查两级指针有效性

☞ 例：`if (p2 != NULL && *p2 != NULL) { **p2 = 10; }`

- ❖ 内存释放顺序：先释放底层内存，再释放上层内存

☞ 例：`free(matrix); // 使用后仅释放行指针`

`// 错误：列内存未释放，导致内存泄漏`

- ❖ 类型一致性：确保二级指针类型与一级指针类型完全匹配、

☞ 例：`int *p1; char **p2 = (char **)&p1;`

`// 错误：类型不匹配，可能导致访问错误`



多级指针

- ❖ 理论上：C 语言支持无限级的指针（ANSI C 标准要求实现支持至少12级指针），如 `int ***p3; int ****p4; ... ; int *****p5; ...`
- ❖ 现实情况：在实际工程中，三级指针已经非常少见超过三级的指针往往意味着程序设计过于复杂，建议重新审视架构
- ❖ 三级指针是 指向二级指针的指针，它存储的是二级指针的内存地址。在处理三维数组、链表的链表等复杂数据结构时具有不可替代的作用。

```
int num = 10;
int *p1 = &num;           // 一级指针p1指向num
int **p2 = &p1;            // 二级指针p2指向p1
int ***p3 = &p2;           // 三级指针p3指向p2
// *p3: 获取 p3 指向的二级指针 p2 的值（即 p1 的地址）
// **p3: 通过 p2 的值（地址）获取一级指针 p1 的值（即 num 的地址）
// ***p3: 通过 p1 的值（地址）获取 num 的实际值
printf("%d\n", ***p3);    // 输出10
```



多级指针（例一）

❖ 例：动态创建2层3行4列的三维数组（张量）

// 分配三维数组（张量）内存

```
void allocate_3d_array(int ***arr, int x, int y, int z, int v) {  
    *arr = (int **)malloc(x * sizeof(int **)); // 分配层指针数组  
    for (int i = 0; i < x; i++) {  
        (*arr)[i] = (int **)malloc(y * sizeof(int *)); // 分配行指针数组  
        for (int j = 0; j < y; j++) {  
            (*arr)[i][j] = (int *)malloc(z * sizeof(int)); // 分配列指针数组  
        }  
    }  
    for (int i = 0; i < x; i++) { // 初始化数组  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                (*arr)[i][j][k] = v;  
            }  
        }  
    }  
}
```



多级指针（例一续）

❖ 例：动态创建2层3行4列的三维数组（张量）

```
// 释放三维数组（张量）内存
void free_3d_array(int ***arr, int x, int y) {
    for (int i = 0; i < x; i++) {
        for (int j = 0; j < y; j++) {
            free((*arr)[i][j]); // 释放列指针数组
        }
        free((*arr)[i]); // 释放行指针数组
    }
    free(*arr); // 释放层指针数组
    *arr = NULL; // 置为空指针
}

int main() {
    int ***tensor;
    allocate_3d_array(&tensor, 2, 3, 4, 0);
    // 使用tensor...
    free_3d_array(&tensor, 2, 3);
    return 0;
}
```



多级指针（例二）

❖ 例：创建复杂的数据结构

```
typedef struct {  
    int id;  
    char *name;  
} Person;  
  
typedef struct {  
    Person **people; // 二级指针，指向Person数组  
    int count;  
} Team;  
  
typedef struct {  
    Team ***teams; // 三级指针，指向多个Team数组  
    int team_count;  
} Organization;
```



多级指针（例二续）

❖ 例：创建复杂的数据结构

```
// 初始化Organization
```

```
Organization init_organization(int num_teams, int num_people) {  
    Organization org;  
    org.team_count = num_teams;  
    org.teams = malloc(org.team_count * sizeof(Team **));  
    for (int i = 0; i < org.team_count; i++) {  
        org.teams[i] = malloc(num_teams * sizeof(Team *));  
        for (int j = 0; j < num_teams; j++) {  
            org.teams[i][j] = malloc(sizeof(Team));  
            org.teams[i][j]->people =  
                malloc(num_people * sizeof(Person *));  
            org.teams[i][j]->count = num_people;  
        }  
    }  
    return org;  
}
```



多级指针的安全使用原则

❖ 初始化规则

- ❧ N级指针必须指向有效的N-1级指针地址

❖ N重检查：

- ❧ 使用前检查N级指针、N-1级指针、... 、一级指针的有效性

❖ 内存释放顺序

- ❧ 从最内层到最外层逐层释放内存

❖ 类型一致性：

- ❧ 确保N级指针类型与指向的N-1级指针类型完全匹配

❖ 避免过度嵌套

- ❧ 超过三级的指针嵌套会显著降低代码可读性

❖ 访问校验

- ❧ 使用宏或函数进行安全访问校验



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化

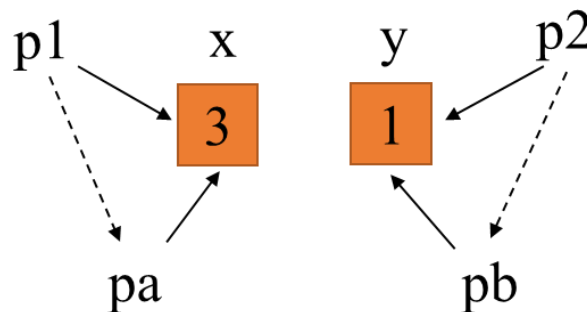


带指针参数的函数

- ❖ 当指针作为函数的参数时，所传的是值，其内容是地址（也称传地址调用）
 - ☞ 被调函数通过指针访问函数外部的数据——具有数据双向传递的功效

```
#include<stdio.h>
void swap2(int *pa, int *pb) {
    int t;
    t=*pa; *pa=*pb; *pb=t;
}

void main() {
    int x=1, y=3, *p1=&x, *p2=&y;
    swap2(p1, p2);
    printf("%d,%d,%d,%d\n", x, y, *p1, *p2);
}
```





带指针参数的函数（例一）

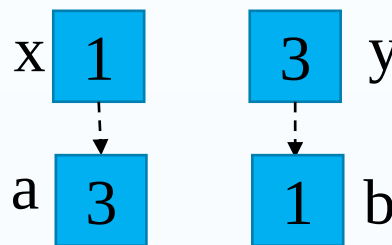
❖ 例：三种变量交换写法的区别

```
void swap1(int a, int b) {  
    int t;  
    t=a; a=b; b=t;  
}
```

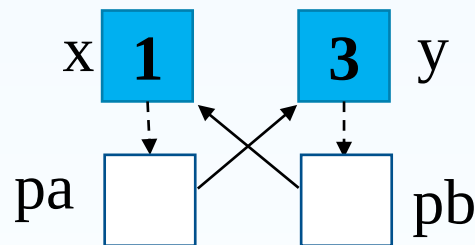
```
void swap2(int *pa, int *pb) {  
    int t;  
    t=*pa; *pa=*pb; *pb=t;  
}
```

```
void swap3(int *pa, int *pb) {  
    int *pt;  
    pt=pa; pa=pb; pb=pt;  
}
```

只交换形参a,b的值，
未修改main函数中的
变量值



交换了形参pa,pb指
针的值，未修改pa,
pb所指变量的值





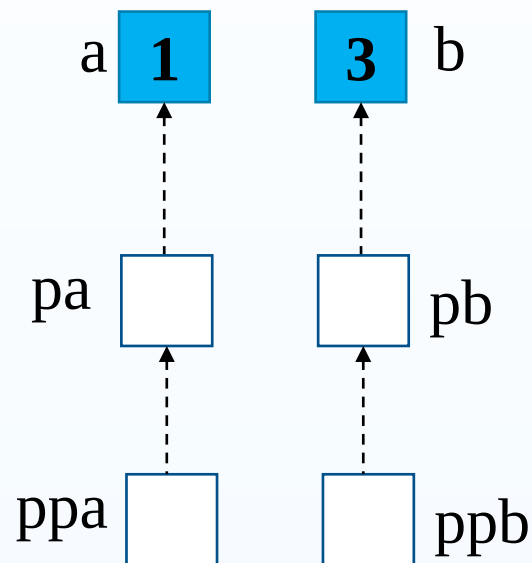
带指针参数的函数（例一续）

❖ 例：三种变量交换写法的区别

```
void swap1(int **ppa, int **ppb) {  
    int t;  
    t=**ppa; **ppa=**ppb; **ppb=t;  
}
```

```
void swap2(int **ppa, int **ppb) {  
    int *pt;  
    pt=*ppa; *ppa=*ppb; *ppb=pt;  
}
```

```
void swap3(int **ppa, int **ppb) {  
    int **ppt;  
    ppt=ppa; ppa=ppb; ppb=ppt;  
}
```



```
int a = 1, b = 3;  
int *pa = &a  
int *pb = &b;  
int **ppa = &pa;  
int **ppb = &pb;
```



带指针参数的函数（例二）

❖ 例：借助void实现编写通用的数据交换函数（泛型编程）

```
void genswap(void *a, void *b, int size) {  
    char t, *ac, *bc;  
    int i;  
    ac = (char *)a; bc = (char *)b;  
    for (i = 0; i < size; i++, ac++, bc++) {  
        t = *ac;  
        *ac = *bc;  
        *bc = t;  
    }  
}
```

```
int a, b; genswap(&a, &b, sizeof(a));  
double c, d; genswap(&c, &d, sizeof(c));  
int e[10], f[10]; genswap(e, f, sizeof(e));  
struct Student g, h; genswap(&g, &h, sizeof(g));
```



带指针参数的函数

- ❖ 有时传指针的目的不是为了让函数修改原变量，而是为了节省传递的数据量（避免值拷贝），实际只是读取变量的值
 - ⌘ 用`const`来保护指针所指向的变量不被修改
 - ⌘ 例如：`void f(const T *p)`；函数内不得修改p所指向对象的值，有时称p是指向常量的指针，常用于T是一个大结构体传值的场景
 - ⌘ 对比：`void f(T * const p)`；是指函数内不得修改指针p的指向，但对p指向的对象没有约束，既可修改该对象的值
 - ⌘ 对比：`void f(const T * const p)`；在函数内指针p的指向和指针p所指向对象的值都不得修改
- ❖ 帮助编译器检查函数内部是否有非法的（设计时禁止）操作



带指针参数的函数（例）

```
typedef struct {  
    char name[20]; int age; float score; char record[1024]; ...  
} Student; //一个很大的结构体  
  
Student update_score_bad(Student s, float new_score) {  
    s.score = new_score;  
    return s; //两次结构体值拷贝  
}  
  
void print_student2_bad(Student s) { //一次结构体值拷贝  
    printf("Name: %s | Age: %d | Score: %.1f\n",  
        s.name, s.age, s.score);  
}  
  
void update_score_good(Student *ps, float new_score) { //指针拷贝  
    if (ps != NULL) ps->score = new_score;  
}  
  
void print_student_good(const Student *ps) { //常量对象的指针拷贝  
    if (ps != NULL) printf("Name: %s | Age: %d | Score: %.1f\n",  
        ps->name, ps->age, ps->score);  
}
```



可变参数函数（例一）

- ❖ VA_LIST宏定义的例子：在任意个整数参数中找出最大值

```
#include <stdarg.h>
```

```
int max_int(int n, ...) {    //...表示参数可变
    va_list ap;              //定义参数列表，实质是一个指针
    int i, current, largest;
    va_start(ap, n);         //初始化，指出可变长度部分开始的位置
    largest=va_arg(ap, int);  //取回单个变量
    for (i=1; i<n; i++) {
        current=va_arg(ap, int); //取回单个变量
        if (current>largest) largest=current;
    }
    va_end(ap);              //结束获取
    return largest;
}
```

```
max_int(3, 28, 35, 13); // 返回 35
```

```
max_int(5, 28, 35, 13, 65, 42); // 返回 65
```




可变参数函数（例二）

❖ VA_LIST宏定义的例子：自定义打印接口的实现

```
int my_printf(const char *fmt, ...) { //...表示参数可变
    va_list args;                    //定义参数列表，实质是一个指针
    va_start(args, fmt);             //初始化，指出可变长度部分开始的位置

    //使用vsprintf一次性读取
    static char gc_PrintfOut[1000];
    vsprintf((char *) gc_PrintfOut, 1000, (char *) fmt, args);
    va_end(args); //结束获取

    puts((const char *)gc_PrintfOut); //使用
    return 0;
}
```

自定义函数 `my_printf("Hello %d", 2026)` 的使用方式与 `printf` 相同



命令行参数的实现

❖ 给 main 函数添加参数，可以实现命令行参数

```
int main(int argc, char *argv[]) { }
```

```
int main(int argc, char **argv) { }
```

☞ argc 代表传递给main的形参的个数，其值至少为1

☞ argv 是指向字符串的指针数组。第一个字符串是程序执行文件名本身，第二个开始是真实参数

☞ 这两个参数由操作系统传递给 main 函数

☞ 例：> gcc -o myprograme main.c



命令行参数的实现（例）

❖ 例：命令行参数获取

```
#include <stdio.h>

void main(int argc, char *argv[]) {
    int i = 1;
    while (i < argc)
        printf("%s%c", argv[i++], (i + 1 < argc ? ' ' : '\n'));
}
```

```
#include <stdio.h>

void main(int argc, char **argv) {
    int i = 1;
    while (i < argc)
        printf("%s%c", *(++argv), (i + 1 < argc ? ' ' : '\n'));
}
```

```
./a.exe -o main main.c
// 输出 -o main main.c
```



带指针返回值的函数

- ❖ 函数的返回值可以是**指针**（即对象的地址），称为指针函数
声明形式：**类型名 *函数名(参数表);**
如：**double *func(int x, double y);**
- ❖ 指针函数的用法同一般函数，返回的指针指向：
 - ∞ 字符串常数，OK，存在静态数据区
 - ∞ 外部全局变量，OK，存在全局数据区
 - ∞ 内部静态变量，OK，存在全局数据区
 - ∞ malloc分配变量，OK，存在堆区需手动free
 - ∞ 调用函数的局部动态变量，OK，存在栈的深处
 - ∞ 被调用函数的局部动态变量，NO，将变为悬空引用



带指针返回值的函数（例）

❖ 一个函数只返回一个值，当需要返回多个值时：

☞ 使用地址值参数传递方式，即调用者提供实参变量的指针；被调函数把返回结果放入相应单元中，如：

```
void fun(int *a, int *b, int n) {  
    b[0]=a[0]; b[1]=a[n-1];  
}
```

☞ 或者用指针返回多个值所在存储空间地址，如：

```
int *fun(int *a, int n) {  
    static int b[2];  
    b[0]=a[0]; b[1]=a[n-1];  
    return b;  
}
```

与外部全局变量的区别是作用域不同



函数指针

- ❖ 函数语句存储在一片连续的内存单元中（代码区），其首地址就是函数的入口地址，函数名则代表该地址
- ❖ 主函数在调用子函数时，就是让程序从主函数转移到子函数的入口地址去执行
- ❖ 程序也可以自定义指针，指向函数的入口地址，即指向函数，以此实现函数调用
- ❖ 定义形式：类型 (*指针变量名)(参数表);
 - ∞ 其中类型即函数返回值类型，参数表即函数形参
 - ∞ 如：float (*func)(int, char, float); 注意与
float *func(int x, char y, float z); 的区别



函数指针

- ❖ 和任何一个指针一样，指向函数的指针也**必须先初始化**（指向某个函数），然后才能使用

如： `double square(double x);` //函数声明

`double (*p)(double x);` //函数型指针声明

`p=square;` //指针初始化

`s=p(1.6);` 或 `s=(*p)(1.6);` //函数调用

☞ 指向的函数的返回值类型和形参必须与指针变量中的定义相一致

- ❖ 整个函数如果需要作为参数在其它函数间进行传送，必须使用指向函数的指针变量作为参数

☞ 函数指针的主要作用是编写通用程序



函数指针（例一）

❖ 例：两个整数的四则运算（回调机制，动态调用）

```
int add(int a, int b) {  
    return a + b;  
}  
  
int( * func_ptr)(int, int) = add; // 定义函数指针  
printf("%d\n", func_ptr(3, 5)); // 输出8  
  
// 作为函数形参  
void process(int a, int b, int( * cb)(int, int)) {  
    printf("Result: %d\n", cb(a, b)); // 实现回调机制 (callback)  
}  
process(4, 6, add); // 输出Result: 10  
  
typedef int( * MathFunc)(int, int);  
MathFunc operations[] = { // 定义数组  
    add, subtract, multiply, divide };  
int result = operations[0](10, 5); // 动态调用, 调用add函数
```




函数指针（例二）

❖ 例：计算定积分

∞ 积分计算函数 `double collect(double a, double b, double prec, double(*pf)(double x) );`

❖ a、b是积分上下限，prec是精度，*pf是被积分函数

∞ 被积分函数 `double func(double x)`

❖ 对自变量 x 求值

∞ 在collect中需要计算被积分函数值时，调用 `(*pf)(x)`

∞ 原理是将积分区间 `[a, b]` 分成 `n` 个小梯形，每个梯形的面积近似代替曲线下的面积，其基本公式为 `( h = (b-a)/n )`：

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right]$$



函数指针（例二续）

❖ 例：计算定积分：为了高效实现，采用变步长梯形法

```
double collect(double a, double b, double prec, double (*pf)(double x)) {  
    int n = 1;           // 初始区间数  
    double h = b - a;    // 初始步长  
    double s1 = (pf(a) + pf(b)) * h / 2.0; // 初始梯形值  
    double s2;           // 下一次迭代的积分值  
    while (n < 1000000) { // 防止死循环：如区间细分过多仍不收敛，则强制退出  
        double sum_new_points = 0.0;  
        // 在现有区间的每个中点处计算新增采样点的值  
        for (int i = 0; i < n; i++)  
            sum_new_points += pf(a + (i + 0.5) * h);  
        // 变步长梯形递推公式：新积分值 = (旧值 + 步长 * 新增点之和) / 2  
        s2 = (s1 + h * sum_new_points) / 2.0;  
        // 如果两次计算结果的差值小于精度要求，则认为收敛  
        if (fabs(s2 - s1) < prec) { return s2; }  
        // 为下一次迭代准备：区间数翻倍，步长减半  
        s1 = s2; n *= 2; h /= 2.0;  
    }  
    return s2;  
}
```



函数指针（例二续）

❖ 例：计算定积分

```
// 测试函数1: x 的平方
double f1(double x) { return x * x; }
// 测试函数2: 正弦函数
double f2(double x) { return sin(x); }

int main() {
    double precision = 1e-6; // 设置精度为 10^-6

    // 计算 x^2 在 [0, 1] 的积分
    double res1 = collect(0, 1, precision, f1);
    printf("Integral of x^2 from 0 to 1: %f\n", res1);

    // 计算 sin(x) 在 [0, PI] 的积分
    double pi = acos(-1.0);
    double res2 = collect(0, pi, precision, f2);
    printf("Integral of sin(x) from 0 to PI: %f\n", res2);
}
```



函数指针（例三）

- ❖ 快速排序库函数 `qsort` 如何在连元素的类型是什么都不知道的情况下，比较两个元素并判断哪个应该在前？
 - ☞ 答案是：`qsort` 函数在执行期间，会通过 `pfCompare` 指针调用 “比较函数”，调用时将要比较的两个元素的地址传给 “比较函数”，然后根据 “比较函数” 返回值判断两个元素哪个更应该排在前面。
 - ☞ 这个 “比较函数” 是由使用 `qsort` 的程序员编写的。在调用 `qsort` 时，将 “比较函数” 的名字作为实参传递给 `pfCompare`。

```
void qsort(void *base, int nelem, unsigned int width,  
int (* pfCompare)(const void *, const void*));
```



函数指针（例三续）

❖ 编写程序，将一个int数组按照个位数从小到大进行排序

```
#include <stdio.h>
#include <stdlib.h>

int my_compare(const void *elem1,
               const void *elem2) {
    int *p1, *p2;
    p1 = (int *) elem1;
    p2 = (int *) elem2;
    return ( *p1 % 10) - ( *p2 % 10);
}
```

```
#define NUM 5

int main() {
    int an[NUM]={8,123,11,10, 4};
    qsort(an, NUM, sizeof(int),
          my_compare);

    for (int i=0; i<NUM; i++)
        printf("%d ", an[i]);
    return 0;
}
```

上面程序的输出结果是：10 11 123 4 8



函数指针（例四）

- ❖ 快速排序库函数 `qsort` 类似的另一个常用库函数是二分查找库函数 `bsearch` （二分查找的前提是数组必须有序）

```
void *bsearch(const void *key, const void *base, int nelem, unsigned  
int size, int (*pfCompare)(const void *, const void *)) ;
```

```
#include <stdlib.h>
```

```
int an[] = { 5, 20, 29, 32, 63 }; // 数组必须有序排列
```

```
int key = 32;
```

```
int my_compare(const void * a, const void * b) {
```

```
    return (*(int*)a - *(int*)b);
```

```
}
```

```
int *item = (int*) bsearch(&key, an, 5, sizeof(int), my_compare);
```



函数指针（例四续）

❖ 二分查找库函数 bsearch 的实现

```
void *bsearch(const void *key, const void *base, size_t nmemb,
              size_t size, int (*compar)(const void *, const void *)) {
    size_t start = 0, end = nmemb;
    while (start < end) {
        // 计算中间索引，防止 (start + end) 溢出
        size_t mid = start + (end - start) / 2;
        // 核心点：计算中间元素的内存地址：
        // 将 base 转换为 char* 以后，偏移 (mid * size) 个字节
        const void *mid_ptr = (const char *)base + (mid * size);

        int res = compar(key, mid_ptr); // 调用用户提供的比较函数
        if (res == 0) { return (void *)mid_ptr; } // 找到目标，返回地址
        else if (res < 0) { end = mid; }          // 目标在左侧
        else { start = mid + 1; }                 // 目标在右侧
    }
    return NULL; // 未找到
}
```



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



动态内存分配

- ❖ 动态内存分配，是在运行时动态申请一段内存空间，并通过指针的类型转换将其视为一个特定对象来使用

- ☞ 常用于创建动态数组、数据结构的结点

- ❖ 申请空间：内存分配函数

```
#include <stdlib.h>
```

```
void *malloc(unsigned int size);
```

- ☞ 分配成功返回指向空类型void的指针，失败返回NULL

- ❖ 常用调用方法（类型转换、大小计算）

```
int *p; p=(int *) malloc(n*sizeof(int));
```

```
if (p == NULL) exit(-1); // 处理分配失败情况
```

unsigned int的大小是四字节，
最大可申请 2^{32} 字节/4GB空间



动态内存分配

❖ 申请空间：内存分配函数

```
void *calloc(size_t nmemb, size_t size);
```

- ☞ 为nmemb个元素的数组分配连续的内存空间，其中每个元素的长度都是size个字节
- ☞ 分配成功返回指向空类型void的指针，并将所有字节初始化为0；失败返回NULL

❖ 分配空间后，可以用 realloc 调整大小

```
void *realloc(void *ptr, size_t size);
```

- ☞ 为ptr指向的动态分配内存重新分配为大小为size个字节的空间。新尺寸可能会大于或小于原有尺寸
- ☞ 分配成功返回指向空类型void的指针，失败返回NULL
- ☞ ptr必须是通过malloc或calloc获得指针



动态内存分配（补充说明）

❖ **realloc** 的行为基于原内存块后续堆空间的情况：

- ❧ **原地扩大（追加）**：如果原内存块后有足够的空闲内存，**realloc** 直接在原内存块后追加空间，内存位置不变，直接返回原指针
- ❧ **异地扩容（重新分配与拷贝）**：如果原内存块后空间不足，**realloc** 会在堆中寻找一块满足要求的足够大的新空间
 - ❖ 将旧内存块中的数据自动拷贝到新空间中（大数据拷贝可能影响性能）
 - ❖ 释放旧的内存块（这步是系统自动完成的，但需注意引用更新）
 - ❖ 返回指向新内存块的指针（若发生异地扩容，原有指针将无效）
- ❧ **缩小空间**：如果新大小小于原大小，**realloc** 可能会减少原内存块的大小，通常原指针不变，部分数据可能会被截断或丢弃



动态内存释放

- ❖ 释放空间：动态分配的内存在使用完毕后应及时释放

```
#include <stdlib.h>
```

```
void free(void *pb);
```

- ❖ 常用调用方法

```
free(p);
```

```
p=NULL; // 防止野指针，养成好习惯
```

- ❖ 注意：

- ❧ free只能释放malloc分配的内存，不能对随便一个指针都应用free

- ❧ 首地址保存好，否则内存无法释放，将造成内存泄漏



动态内存分配（例一）

❖ 如果内存释放过早，可能造成悬空引用

☞ 必须谨慎处理指向malloc分配空间的指针

```
void main() {  
    int *a=(int *)malloc(40);  
    .....  
    free(a);  
    .....  
    ...*(a+5)...;  
    .....  
}
```

```
int *fun() {  
    int a=5;  
    return(&a);  
}  
  
void main() {  
    int *b;  
    b=fun();  
    ...*b...;  
    .....  
}
```



动态内存分配（例二）

❖ 例：结构体中的可变长数组（**柔性数组**：C99标准）

∞ 动态数组放在结构体末尾，结构体和动态数据在内存中连续存储

```
typedef struct {  
    int length;  
    int items[];  
    // 或者int items[0]; 不占用结构体内存，  
    // 但必须位于最后，之前至少包含一个其他成员
```

```
} Node;
```

```
int n = 10;
```

```
Node *p = (Node *)malloc(sizeof(Node) + n*sizeof(int));
```

```
p->length = n; for(int i=0;i<n;++i) p->items[i] = i + 1;
```

```
free(p); p = NULL;
```

sizeof(Node)=4



柔性数组与指针成员的区别

特性	柔性数组	指针成员
内存布局	紧跟结构体，连续内存	结构体仅存地址，内存不连续
分配次数	1次 (<code>malloc</code>)	2次 (<code>malloc</code> 结构体 + <code>malloc</code> 数组)
释放次数	1次 (<code>free</code>)	2次 (<code>free</code> 数组 + <code>free</code> 结构体)
访问效率	高 (直接访问)	低 (一次访存获取地址，再访存获取数据)

注意事项

- 柔性数组属于C99标准，在旧的编译器中可能需要使用 `[0]` 或 `[1]` 形式。
- 不能直接定义结构体数组变量包含柔性成员(如 `struct Node d[10];`)，因为编译器无法确定总体内存大小。



建立动态数组

❖ 建立一维数组

- ❧ 定义指向所需数据类型的**指针变量**
- ❧ 按照指定的长度和数据类型**申请**分配存储空间
- ❧ 将动态分配获取的存储区域首地址**强制类型转换**后赋值给指针变量

❧ 例如：

```
int *pb;  
if ((pb=(int *)malloc(n*sizeof(int)))==NULL) {  
    printf("内存不足\n");  
    exit(1);  
}
```

- ❧ 使用方法与一般指针相同，也可作为一维数组名使用



建立动态数组

- ❖ 建立二维数组：已知数组的第二维大小
 - ❧ 定义二维数组相应的指针变量
 - ❧ 申请相应的内容空间
 - ❧ 首地址强制类型转换

```
int [][10] pb=(int [][10])malloc(5*10*sizeof(int));
```

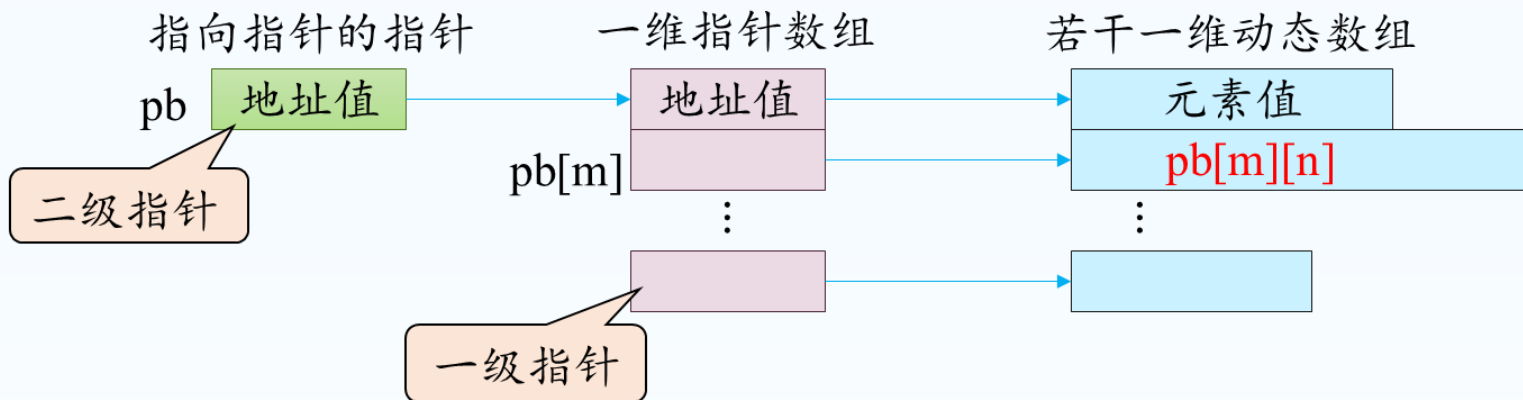
或

```
int (*pb)[10] = (int (*)[10]) malloc(5*10*sizeof(int));
```



建立动态数组

- ❖ 也可以利用指针数组，构造**第二维大小不一致的“数组”**
 - ❧ 定义指向指向所需数据类型的指针的指针变量（二级指针变量）
 - ❧ 以二维数组的行为长度，动态创建一维指针数组，并将其首地址赋值给二级指针变量
 - ❧ 以二维数组的列为长度，动态创建若干个（由行数决定）一维数组，并将其首地址分别赋值给指针数组中的对应元素（一级指针）
 - ❧ 按二维数组方式或指针方式使用二级指针变量





内存池管理

- ❖ 动态内存分配：在标准 C 中，`malloc` 是通过系统调用（如 `brk` 或 `mmap`）向操作系统申请内存的，在高性能应用中存在以下问题：
 - ❧ 性能开销：频繁的系统调用会导致用户态和内核态之间的切换，浪费 CPU 周期，系统调用的开销在每秒万次级的操作下会变得极其沉重
 - ❧ 内存碎片：频繁分配和释放大小不一的内存块，会导致物理内存中出现大量无法利用的小空隙，最终无法申请到连续的大块内存
 - ❧ 内存泄漏风险：生命周期管理的疏忽会导致程序占用内存持续攀升
- ❖ 内存池管理：预先申请一大块连续内存，然后在程序内部自行管理这块内存的分配和回收，用以满足后续所有的内存需求
- ❖ 内存池的两种主流模型：
 - ❧ 固定大小内存池：预先划分为大小相等的块，效率最高，逻辑最简单。
 - ❧ 变长内存池：允许分配不同大小的内存块，实现较复杂



内存池管理

❖ 内存池的优缺点

维度	内存池	标准 malloc
分配速度	极快 ($O(1)$)	较慢 (搜索链表)
内存碎片	极少 (块固定)	严重 (产生大量空洞)
管理复杂度	较高 (需手动维护池)	简单 (系统托管)
适用场景	高频、固定大小分配	随机、不确定大小分配

❖ 内存池的核心步骤：

- ❧ 初始化：申请一大块内存，并将其切割成一个个相同大小的块，并串起来
- ❧ 分配：从内存池头部取出一个块，并将头指针指向下一个空闲的内存块
- ❧ 释放：将用完的块插回内存池的头部（操作时间复杂度为 $O(1)$ ）



内存池管理（例）

❖ 例：基于数组的静态内存池

∞ 内存池定义

```
#include <stdio.h>
#include <stdint.h>
#define POOL_SIZE 10      // 内存池中块的数量
#define BLOCK_SIZE 32    // 每个内存块的大小（字节）

typedef struct {
    // 实际存储数据的二维数组
    uint8_t data[POOL_SIZE][BLOCK_SIZE];
    int next_free[POOL_SIZE]; // 存储下一个空闲块索引的数组
    int first_free;           // 当前第一个可用的空闲块索引
    int free_count;           // 剩余空闲块数量
} StaticPool;
```

☑ 零碎片化：因为所有块的大小都是固定的（BLOCK_SIZE），所以永远不会出现“有空间但不够连续”的碎片化情况。



内存池管理（例）

❖ 例：基于数组的静态内存池

🌀 初始化内存池

```
void pool_init(StaticPool *pool) { // 初始化内存池
    for (int i = 0; i < POOL_SIZE - 1; i++) {
        pool->next_free[i] = i + 1; // 每个块指向下一个
    }
    // 最后一个块标记为结尾
    pool->next_free[POOL_SIZE - 1] = -1;
    pool->first_free = 0;
    pool->free_count = POOL_SIZE;
}
```

☑ 将一个大数组划分为 POOL_SIZE 个固定大小的内存块。为了高效找到哪个内存块是空的，初始化时会在数组内部维护一个索引链表：next_free 数组，每个元素都指向它的下一个邻居（0->1, 1->2 ... 9->-1）



内存池管理（例）

❖ 例：基于数组的静态内存池

∞ 分配内存

```
void* pool_alloc(StaticPool *pool) { // 分配内存
    if (pool->first_free == -1) {
        printf("Error: Pool out of memory!\n");
        return NULL;
    }

    // 获取当前空闲块的指针
    void *res = &(pool->data[pool->first_free]);
    // 更新 first_free 到下一个空闲位置
    pool->first_free = pool->next_free[pool->first_free];
    pool->free_count--;
    return res;
}
```

☑低复杂度：无论是分配还是释放，都只涉及几行简单的赋值操作，速度极快。



内存池管理（例）

❖ 例：基于数组的静态内存池

🔗 释放内存

```
void pool_free(StaticPool *pool, void *ptr) { // 释放内存
    // 根据指针地址计算出它在数组中的索引
    int index = ((uint8_t*)ptr - (uint8_t*)pool->data) / BLOCK_SIZE;
    if (index < 0 || index >= POOL_SIZE) {
        printf("Error: Pointer out of pool bounds!\n");
        return;
    }

    // 将释放的块插回到空闲链表的头部
    pool->next_free[index] = pool->first_free;
    pool->first_free = index;
    pool->free_count++;
}
```

☑ 指针算术的妙用：在 释放内存的过程（pool_free）中，没有使用低效的遍历，而是通过 $(ptr - start) / size$ 这种数学方式直接算出索引。



内存池管理（例）

❖ 例：基于数组的静态内存池

∞ 内存池的使用

```
void main() {
    StaticPool myPool;    pool_init(&myPool);    // 初始化内存池

    int *p1 = (int*)pool_alloc(&myPool); // 申请内存
    if (p1) { *p1 = 42; printf("Allocated p1, value: %d\n", *p1); }

    char *p2 = (char*)pool_alloc(&myPool); // 申请内存
    if (p2) {
        snprintf(p2, BLOCK_SIZE, "Hello Memory Pool!");
        printf("Allocated p2, content: %s\n", p2);
    }

    printf("Remaining blocks: %d\n", myPool.free_count);
    pool_free(&myPool, p1); // 释放内存
    printf("After free p1, remaining blocks: %d\n", myPool.free_count);
}
```



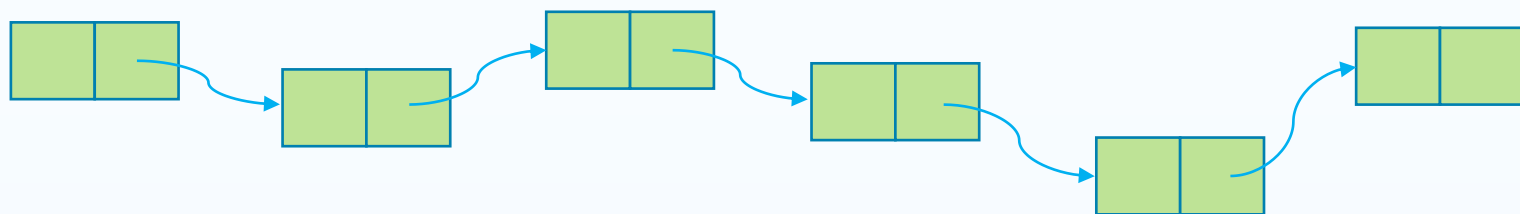
本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



链表的定义

- ❖ 链表是一种动态管理的数据结构
 - ❧ 用结构体类型定义复杂数据结构的结点
 - ❧ 用结构体指针将若干个结点串连在一起
 - ❧ 用动态分配空间的方式产生实际数据结点



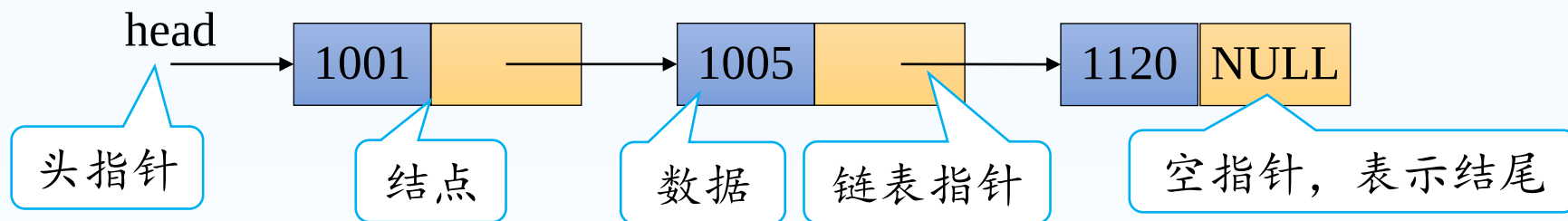
- ❖ 数组的分量称为元素，链表的分量称为结点，一个结点就是一个结构体类型的数据
- ❖ 链表可以方便地实现插入、删除操作



链表的定义（例）

- ❖ 例：任务链表，将一系列任务连成一个表。其数据结点包括：
 - ∞ 任务的内容/索引
 - ∞ 指向下一个数据结点的指针

```
struct Node {  
    int task_num;  
    struct Node *next;  
};
```





链表的创建

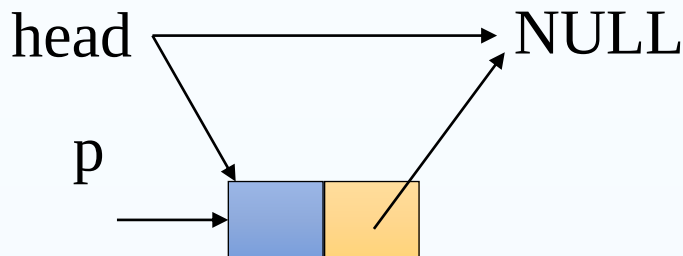
❖ 链表初始化

```
struct Node *head;  
head=NULL;
```

head → NULL

❖ 创建链表的第一个结点:

```
struct Node *p;  
p=(struct Node *) malloc(sizeof(struct Node));  
p->next=head;  
head=p;
```



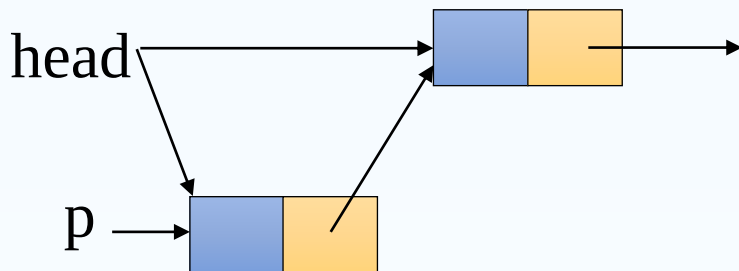


链表的创建

❖ 头插法：在链表头顺序插入新结点来建立链表

```
struct Node *p;  
p=(struct Node *) malloc(sizeof(struct Node));  
p->next=head;  
head=p;
```

∞ 先插入的在链表尾，后插入的在链表头



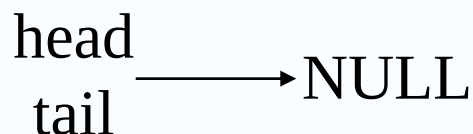


链表的创建

❖ 尾插法：在链表尾顺序插入新结点来建立链表

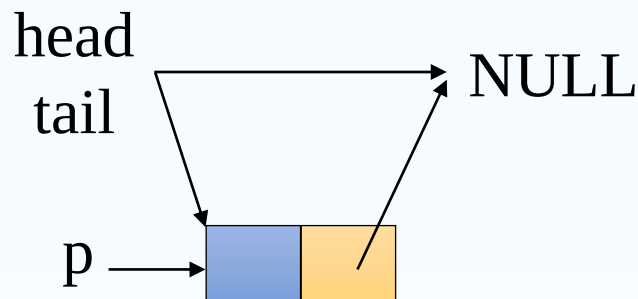
∞ 除头指针head外，再增加一个尾指针tail，并进行初始化

```
struct Node *head, *tail;  
head=tail=NULL;
```



∞ 插入第一个结点

```
struct Node *p;  
p=(struct Node *) malloc(  
    sizeof(struct Node));  
p->next=NULL;  
head=tail=p;
```



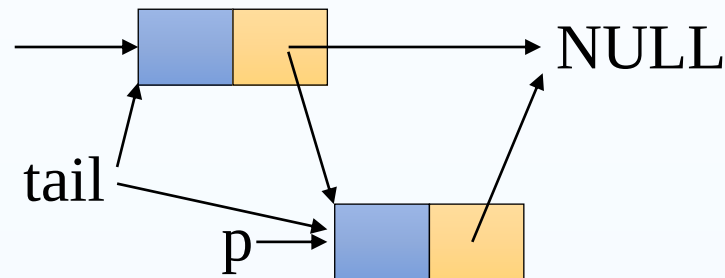


链表的创建

❖ 尾插法：在链表头顺序插入新结点来建立链表

∞ 插入新结点

```
struct Node *p;  
p=(struct Node *) malloc(  
    sizeof(struct Node));  
p->next=NULL;  
tail->next=p;  
tail=p;
```



∞ 先插入的在链表头，后插入的在链表尾



链表的遍历

- ❖ 用一个指针指向链表头（初始化）
- ❖ 将指针指向下一个结点（移动），直至找到目标节点（值），或达到链表末尾（ $q \rightarrow \text{next} == \text{NULL}$ ）
- ❖ 读取指针所指向结点的内容（访问）

```
struct Node *lookup(struct Node *head, int k) {  
    struct Node *q;  
    q=head;  
    while (q!=NULL && q->num!=k)  
        q=q->next;  
    if (q==NULL) return(NULL);  
    else return(q);  
}
```



链表的插入

- ❖ 在链表头部插入一个新结点（头插法）
- ❖ 在链表任意位置（除链表头外）插入一个新结点

```
struct Node *p, *q;
```

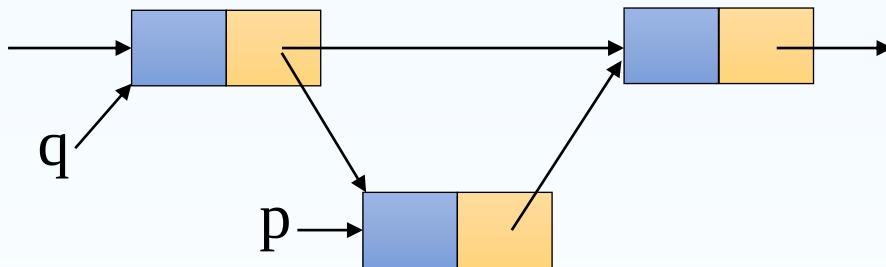
```
// 找到待插入位置q（新结点将插入在q之后）;
```

```
p=(struct Node *) malloc(sizeof(struct Node));
```

```
p->next=q->next;
```

```
q->next=p;
```

∞ 要注意不要丢失连接：先连接后续结点，再接入前继结点





链表的删除

❖ 链表结点的删除

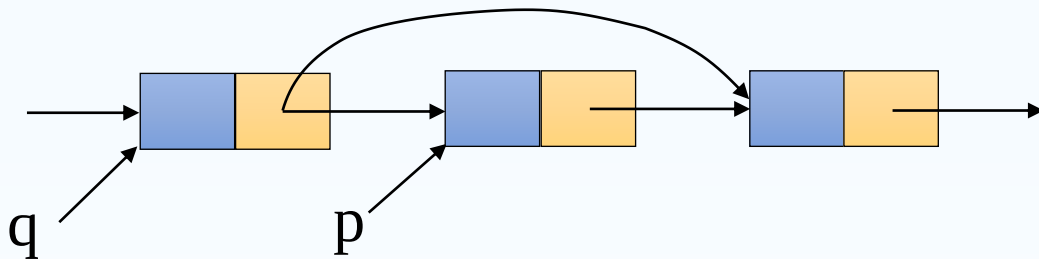
```
struct Node *p,*q;
```

```
// 遍历查找链表，找到待删除结点的指针p，以及p的前一个结点指针q;
```

```
q->next=p->next;
```

```
free(p);
```

☞ 删除首结点时，代码应改为 $\text{head} = p \rightarrow \text{next}$;





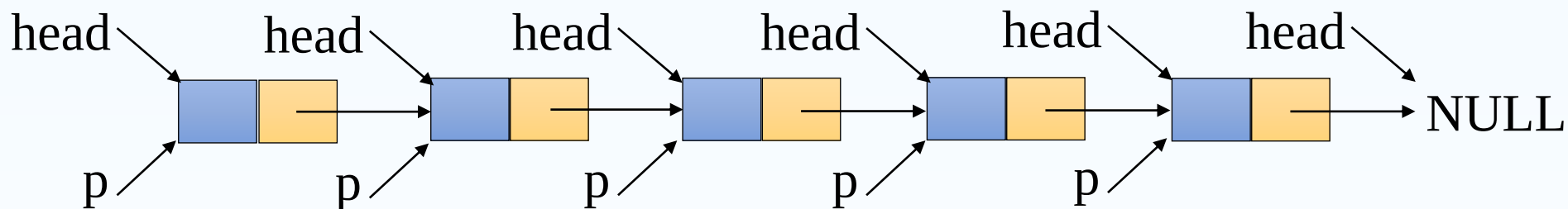
链表的删除

❖ 整个链表删除

☞ 从链表的尾部开始，逐个结点向前删除

☞ 从前向后删除

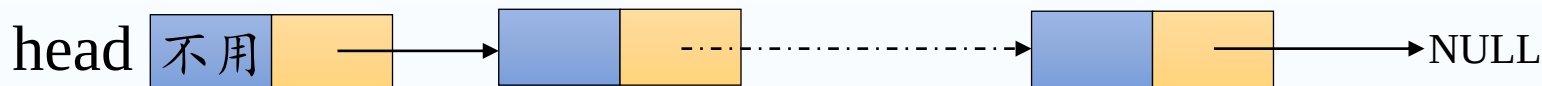
```
struct Node *p;  
while (head!=NULL) {  
    p=head; head=head->next; free(p);  
}
```





链表的改造: 虚拟首结点

- ❖ 在链表的任意位置插入与在链表头插入的过程不一样
 - ∞ 一般插入点的前面是结点，表头的前面是head指针
- ❖ 虚拟首结点：第一个结点不用



- ∞ 好处：只需调用一般位置插入即可，无需区分是否是首结点插入

- ∞ 链表初始化

```
struct Node head;
```

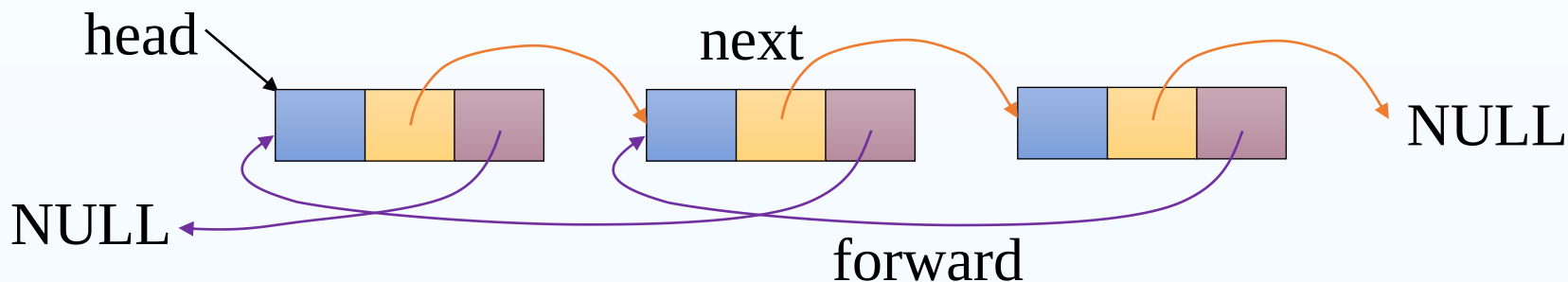




链表的改造: 双向链表

- ❖ 访问链表时，经常会遇到“求上一个结点”的问题

```
struct Node {  
    int num;  
    struct Node *next, *forward;  
};
```

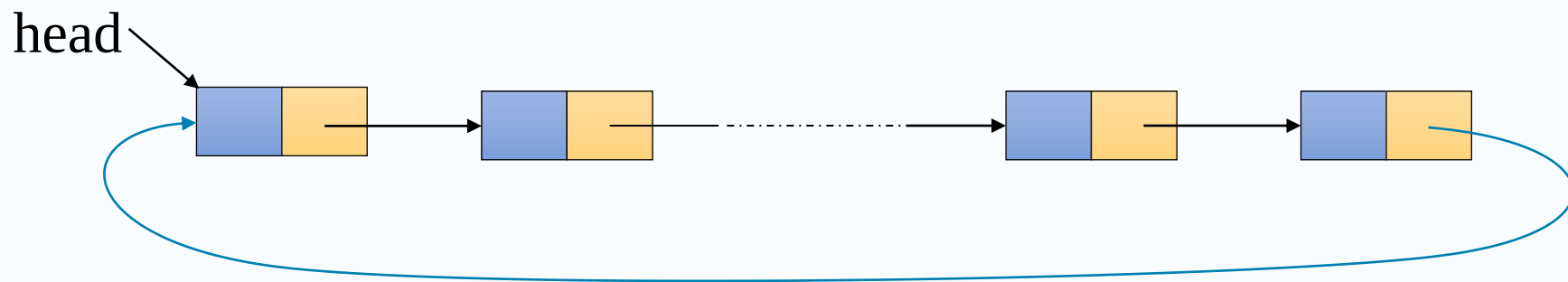


- ∞ 优点：节省查询时间；算法更灵活
- ∞ 代价：要付出多一倍的指针存储空间



链表的改造: 循环链表

❖ 将链表尾的指针指回链表开始

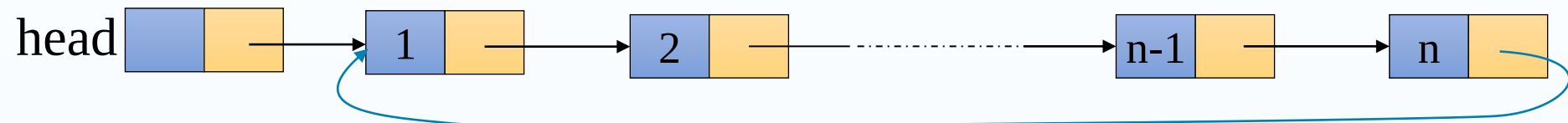


∞ 优点: 适用于处理需要循环访问的场景



链表的改造: 循环链表 (例)

- ❖ 例 (约瑟夫问题): 有 n 个人, 编号为 $1 \sim n$, 从第一个人开始从1到 m 报数, 报到 m 的人出局, 然后从第 $m+1$ 个人开始, 重复以上过程。问最后一个人的编号。



```
#include <stdlib.h>
#include <stdio.h>
struct Node {
    int num;
    struct Node *next;
};
```

```
void main() {
    int n=20, m=3, i, j;
    struct Node head, *p, *q;
    p=&head;
    for (i=1; i<=n; i++) { //依次插入结点
        p->next=(struct Node *) malloc(
            sizeof(struct Node));
        p=p->next; p->num=i;
    }
    p->next=head.next;
```




链表的改造: 循环链表 (例续)

- ❖ 例 (约瑟夫问题): 有 n 个人, 编号为 $1 \sim n$, 从第一个人开始从1到 m 报数, 报到 m 的人出局, 然后从第 $m+1$ 个人开始, 重复以上过程。问最后一个人的编号。
 - ∞ 让 p 指向当前报数之人, 向后移 $m-2$ 次, 用 q 指向 p 的后续结点 (即出局的人), 删除 q 结点

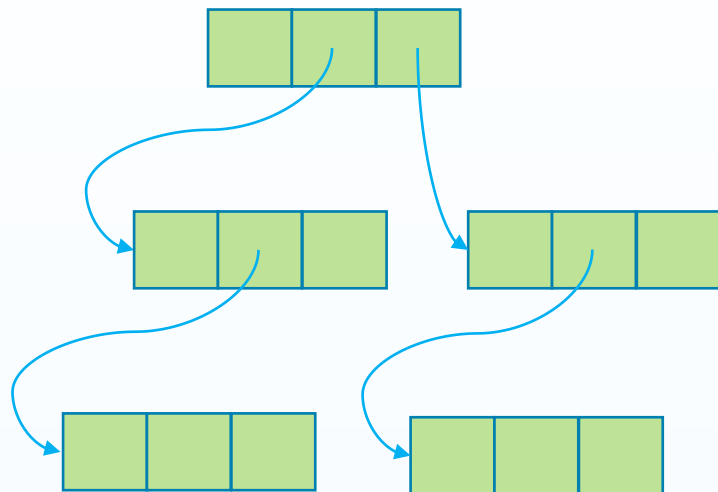
```
p=head.next;
for (i=1; i<n; i++) { //模拟报数过程
    for (j=2; j<=m-1; j++) p=p->next;
    q=p->next; p->next=q->next; p=q->next;
    free(q);
}

printf("%d\n", p->num);
}
```

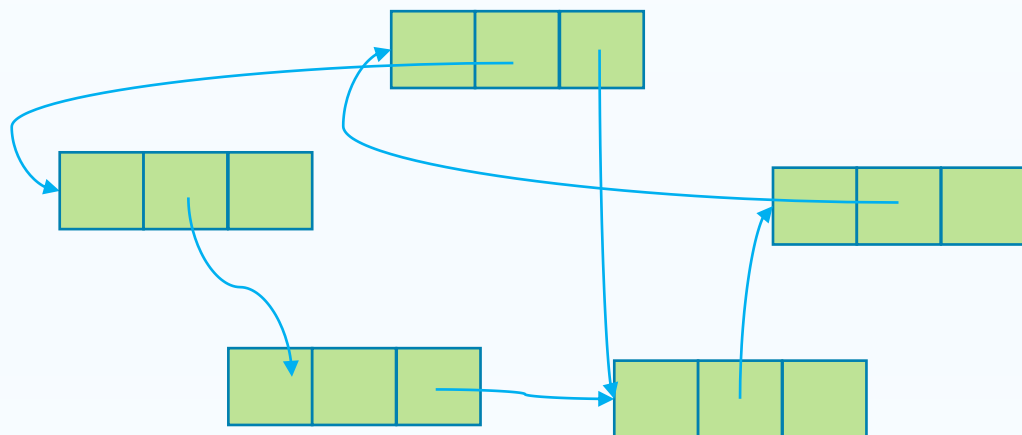


复杂构造数据类型

❖ 树：



❖ 图：结点结构与树类似，但结点间的连接更复杂



❖ 根据问题的不同，设计相应的算法和数据结构



复杂构造数据类型（例一）

- ❖ 设计一个通过自由链表来管理可用的内存块的内存池

```
#include <stdio.h>
#include <stdlib.h>

typedef struct MemoryBlock {
    struct MemoryBlock *next; // 指向下一个空闲块
} MemoryBlock;

typedef struct {
    void *pool_start;          // 内存池首地址
    MemoryBlock *free_list;    // 空闲块链表头
    size_t block_size;        // 每个块的大小
    int total_blocks;          // 总块数
} MemoryPool;
```



复杂构造数据类型（例一）

- ❖ 设计一个通过自由链表来管理可用的内存块的内存池

```
MemoryPool* create_pool(int num_blocks, size_t size) { // 初始化内存池
    MemoryPool *mp = (MemoryPool*)malloc(sizeof(MemoryPool));
    // 确保够放指针
    mp->block_size = size < sizeof(MemoryBlock) ? sizeof(MemoryBlock) : size;
    mp->pool_start = malloc(num_blocks * mp->block_size); // 分配一整块内存
    mp->free_list = (MemoryBlock*)mp->pool_start;

    // 将大块内存按block_size分成小块，并将内存块串成链表
    MemoryBlock *curr = mp->free_list;
    for (int i = 0; i < num_blocks - 1; i++) {
        curr->next = (MemoryBlock*)((char*)curr + mp->block_size);
        curr = curr->next;
    }
    curr->next = NULL; // 最后一个块指向空
    return mp;
}
```



复杂构造数据类型（例一）

- ❖ 设计一个通过自由链表来管理可用的内存块的内存池

```
void* pool_alloc(MemoryPool *mp) { // 分配内存
    if (mp->free_list == NULL) return NULL; // 池子空了

    void *block = mp->free_list;
    mp->free_list = mp->free_list->next; // 移动头指针
    return block;
}

void pool_free(MemoryPool *mp, void *p) { // 释放内存
    MemoryBlock *block = (MemoryBlock*)p;
    block->next = mp->free_list; // 插回链表头部
    mp->free_list = block;
}

void destroy_pool(MemoryPool *mp) { // 销毁内存池
    free(mp->pool_start);
    free(mp);
}
```



系统动态内存管理原理*

- ❖ 利用可利用空间表管理内存：
 - ❧ 建立链表，一个空闲块用一个结点来表示
 - ❧ 用户请求分配时，系统从表中删除一个结点分配之
 - ❧ 用户释放所占内存时，系统即回收并将它插入到表中
- ❖ 最简单的可利用空间表的结构：
 - ❧ 系统运行期间所有用户请求分配的存储量大小相同
 - ❧ 由于表中结点大小相同，则在分配内存时无需查找



系统动态内存管理原理*

❖ 系统运行期间用户请求分配的存储量有若干种大小的规格

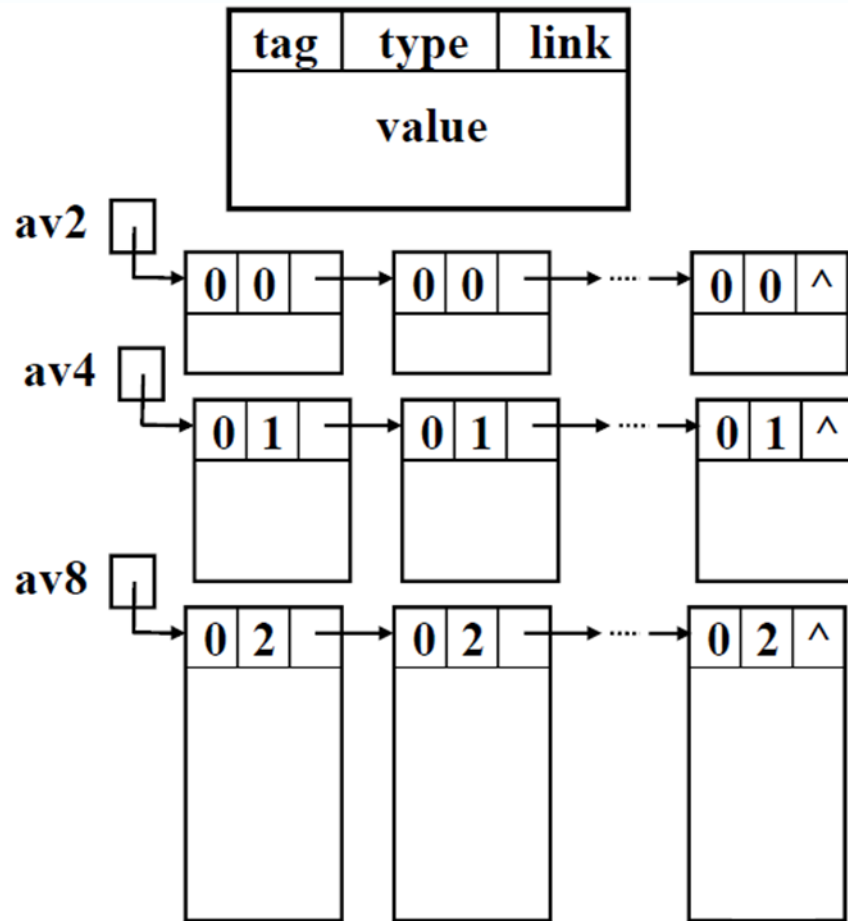
∞ 建立若干个可利用空间表，同一链表中的结点大小相同

∞ 每个结点的第一个字设有

❖ 链域(link): 指向同一链表中下一结点的指针

❖ 标志域(tag): 0-空闲块、1-占用块

❖ 结点类型域(type): 区分大小不同的结点





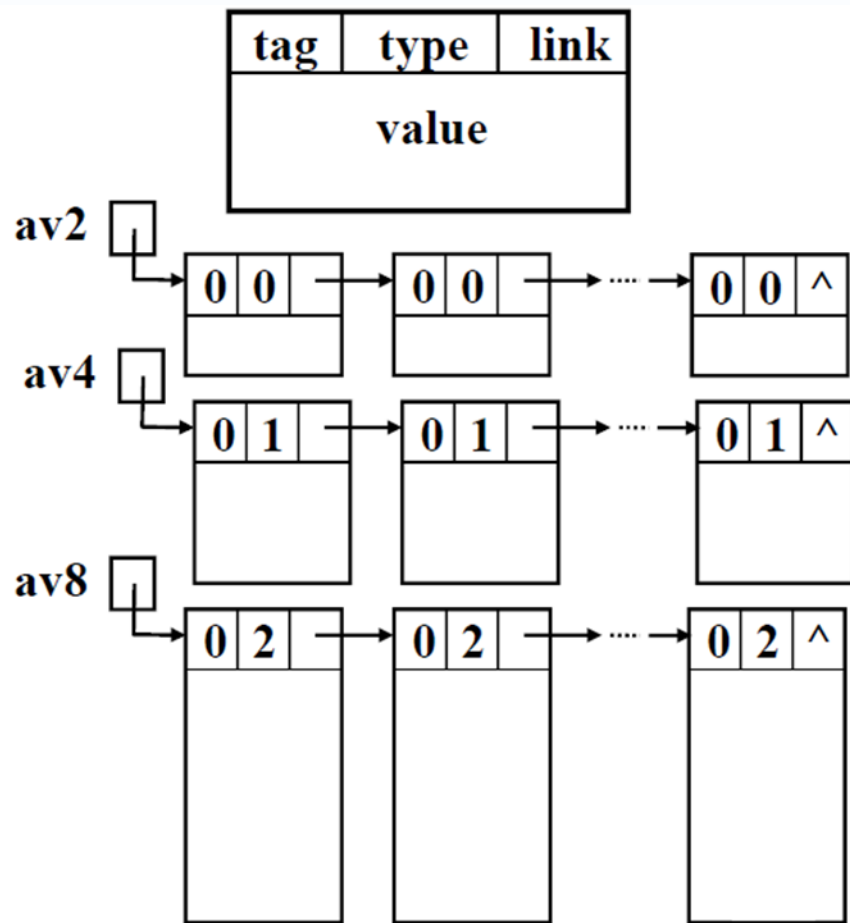
系统动态内存管理原理*

❖ 系统运行期间用户请求分配的存储量有若干种大小的规格

∞ 分配时，执行：

- ❖ 从结点大小和请求分配的量相同的链表中查找结点并分配之
- ❖ 若没有，则从结点较大的链表中查找结点，将其中一部分分配给用户，剩余的插入到相应大小的链表中
- ❖ 若各链表都没有合适的结点，则要执行“存储紧缩”将小块合并

∞ 回收时，将释放的空闲块插入到相应大小的链表的表头



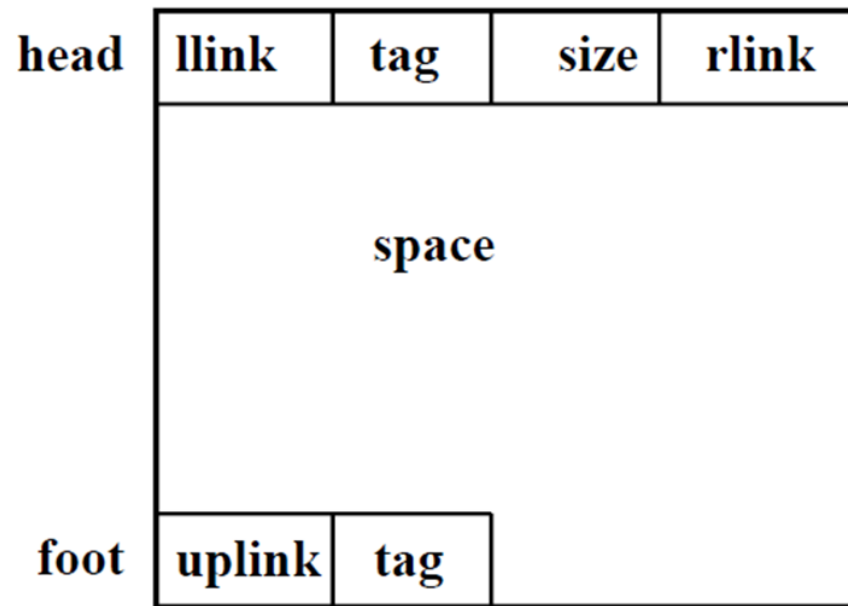


系统动态内存管理原理*

❖ 可利用空间表的数据结构

```
struct WORD{ // 内存字类型
    union {
        WORD *llink; // 指向前驱结点
        WORD *uplink; // 指向本结点头部
    };
    int tag; // 0-空闲; 1-占用
    int size; // 块大小
    WORD *rlink; //指向后继结点
    OtherType other; //字的其它部分
} head, foot, *space;
```

```
// 指向p所指结点的底部
#define FootLoc(p) p+p->size-1
```





复杂构造数据类型（例二）

❖ 为一个图书馆的藏书建立一个按书名检索的系统。

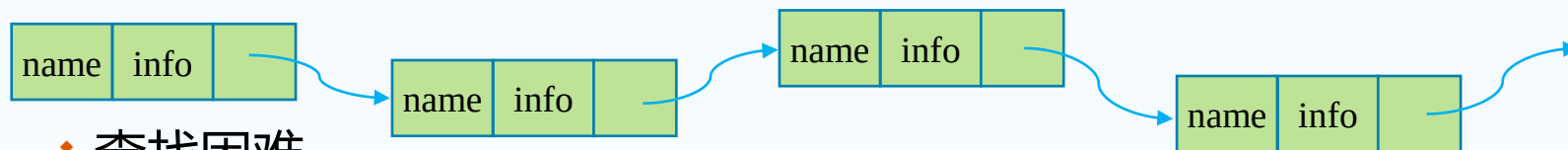
∞ 方案一：建立一个结构体数组

```
struct book_list{  
    char name[101];  
    char info[1001];  
} books[1000];
```

❖ 需要设定书册上限

❖ 配合快速查找算法、插入和删除操作

∞ 方案二：建立一个书籍的链表



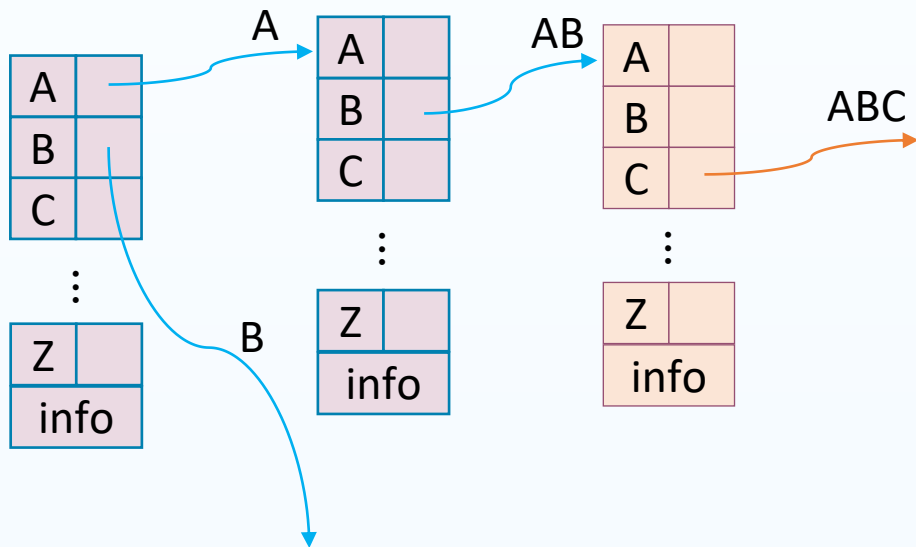
❖ 查找困难



复杂构造数据类型（例二）

❖ 为一个图书馆的藏书建立一个按书名检索的系统。

∞ 方案三：建立一个**书名目录**的特殊数据结构



```
struct book_tree{
    char info[1000];
    struct book_tree *next[26];
};

struct book_tree *head;  ....

char * lookup(struct book_tree *p,
               char *bookname) {
    if (*bookname=='\0')
        return(p->info);
    int c=*bookname-'A';
    if (p->next[c]==NULL)
        return(error);
    return lookup(p->next[
        *bookname-'A'], bookname+1);
}  ....

lookup(head, "ABC");
```

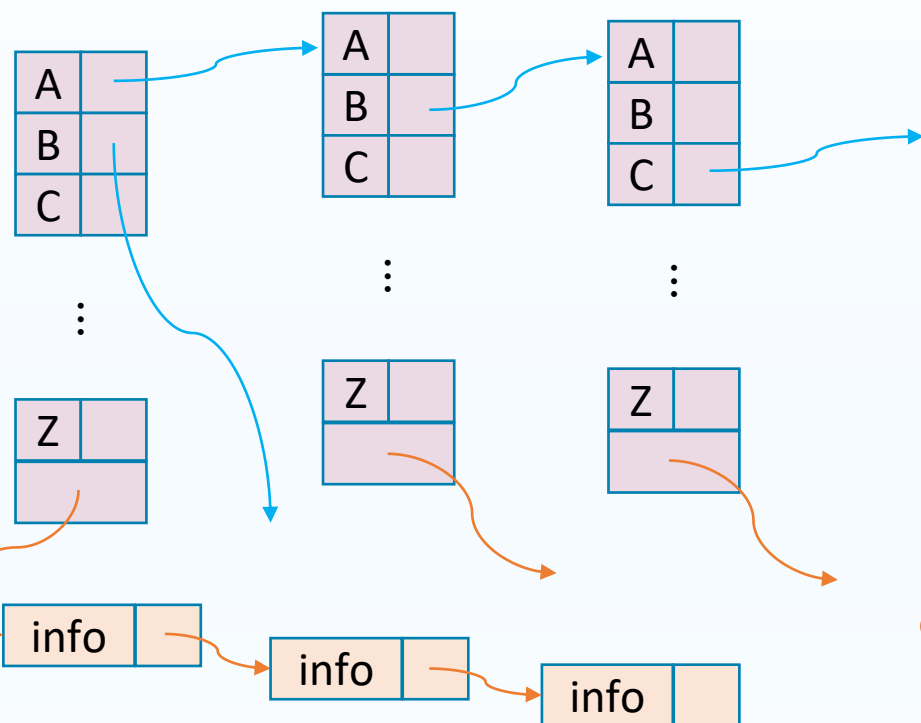


复杂构造数据类型（例二）

❖ 为一个图书馆的藏书建立一个按书名检索的系统。

∞ 建立一个书目的数据结构

∞ 重名书籍的处理：将info部分扩展为一个链表



```
struct book_info{
    char info[1000];
    struct book_info *next;
};

struct book_tree{
    struct book_info *info;
    struct book_tree *next[26];
};
```



链表小结

❖ 数据结构的存储方式有两种， 需要根据需求进行选择：

❧ 数组（顺序存储）：

- ❖ 优点：紧凑连续存储，可以随机访问，通过索引快速找到对应元素，而且相对节约存储空间
- ❖ 缺点：内存空间必须一次性分配够，如果要扩容，需要重新分配一块更大的空间，再把数据全部复制过去；而且如果想在数组中间进行插入和删除，每次必须搬移后面的所有数据以保持连续

❧ 链表（链式存储）：

- ❖ 优点：元素不连续，不存在数组的扩容问题；如果知道某一元素的前驱和后驱，操作指针即可删除该元素或者插入新元素
- ❖ 缺点：无法根据索引算出对应元素的地址，不能随机访问；而且由于每个元素必须存储指向前后元素位置的指针，会消耗相对多的储存空间



链表小结

- ❖ 数据结构的基本操作：
 - ❧ 初始化、销毁（内存回收）
 - ❧ 数据查询：个体访问、整体遍历
 - ❧ 动态变化：插入元素、删除元素
- ❖ 复杂数据处理通常可归结为四个基本操作的组合：
 - ❧ 增： `Insert(&L, i, e)` 在第*i*个位置之前插入新的数据元素*e*
 - ❧ 删： `Delete(&L, i, &e)` 删除第*i*个数据元素，并返回值*e*
 - ❧ 查： `Traverse(L, visit())` 依次对每个元素调用函数*visit()*
 - ❧ 改： `GetElem(L, i, &e)` 用*e*返回*L*中第*i*个数据元素的值



本章内容概述

- ❖ 指针基本概念回顾
- ❖ 指向数组的指针与指针数组
- ❖ 二级与多级指针
- ❖ 带指针的函数与函数指针
- ❖ 高级内存管理技术
 - ❧ 动态内存分配
 - ❧ 内存池管理
- ❖ 应用实例：链表
- ❖ 内存调试与优化



内存调试概述

❖ C语言编程中，常见的五大内存Bug

- ❧ 内存泄漏：`malloc`内存却忘记 `free`，随着时间推移耗尽系统资源
- ❧ 野指针/悬空指针：指向已释放内存的指针
- ❧ 缓冲区溢出：写入的数据超过了数组或分配内存的边界
- ❧ 非法访问：解引用 `NULL` 指针或访问未授权的内存地址
- ❧ 重复释放：对同一块内存调用两次 `free`，会导致堆管理器崩溃

❖ 防御性编程实践：Debug的最高境界是让Bug无法生存

- ❧ 指针初始化：定义指针时立即赋值为 `NULL`。
- ❧ 释放后置空：执行 `free(p)`；后立即执行 `p=NULL`；（防止野指针）。
- ❧ 检查返回值：永远检查 `malloc` 返回的是否为 `NULL`。
- ❧ 使用安全函数：用 `strncpy` 代替 `strcpy`，防止字符串溢出。



内存调试技巧

- ❖ **静态分析**：在代码运行之前，通过编译器或工具检查潜在风险
 - ❧ 编译器警告：gcc开启 `-Wall -Wextra`，编译器能帮你发现未初始化的指针、类型不匹配等初级错误。
 - ❧ Cppcheck：专门针对 C/C++ 的静态工具，它能检测出逻辑上的内存泄漏或越界，即使代码符合语法规范。
- ❖ **运行时调试**：当提示 **Segmentation Fault** (段错误) 时，gdb现场分析
 - ❧ `backtrace (bt)`：当程序崩溃时，输入此命令查看函数调用栈。它能告诉你崩溃发生在哪一行。
 - ❧ `print &variable`：查看变量地址，确认指针是否指向了预期的内存区域。
 - ❧ `watch *0x7fffffffefe000` (监视点)：这是高级技巧。如果你发现某个变量莫名其妙被改了，设置监视点，程序会在该地址内容发生变化的瞬间停下。



GCC 的警告选项

- ❖ -Wall : 涵盖了最常见的、极有可能是 Bug 的代码问题
 - ❧ 未初始化的变量: 使用了尚未赋值的局部变量
 - ❧ 未使用的变量/函数: 定义了但从未使用的变量、函数或参数
 - ❧ 控制流问题: 比如函数应该有返回值但漏掉了 return 语句
 - ❧
- ❖ -Wextra : 针对合法但存在潜在风险或编码风格不佳的情况
 - ❧ 指针符号差异: 比如将 char * 传递给期望 unsigned char * 的函数
 - ❧ 有符号与无符号比较: 比如将 int 和 unsigned int 进行比较
 - ❧ 空的 if/else 语句体: 比如不小心在 if 后面加了分号: if (x > 0);
 - ❧
- ❖ 在实际开发中, 通用的做法是将两者结合使用, 甚至配合 -Werror (将所有警告视为错误) : gcc -Wall -Wextra -Werror main.c -o main



GCC 的警告选项 (例)

- ❖ -Wall 例：返回局部变量的地址 (-Wreturn-local-addr)

```
int* get_data() {  
    int x = 100;  
    return &x;  
}
```

-Wall 提示： warning: function returns address of local variable

// 警告：返回了局部变量的地址 &x

- ❖ -Wall 例：使用未初始化的指针 (-Wuninitialized)

```
void test() {  
    int *p;  
    *p = 50;  
}
```

-Wall 提示： warning: 'p' is used uninitialized

// 警告：使用了未初始化的指针 'p'



GCC 的警告选项 (例续)

- ❖ -Wextra 例: 指针符号不一致 (-Wpointer-sign)

```
void process(unsigned char *data);  
void call() {  
    char *str = "Hello";  
    process(str);  
}
```

-Wextra 提示: warning: pointer targets in passing argument 1 of 'process' differ in signedness
// 警告: 传参时指针符号不一致

- ❖ -Wextra 例: 结构体指针成员缺失初始化 (-Wmissing-field-initializers)

```
struct Node {  
    int id;  
    char *name;  
};  
struct Node n = { .id = 1 };
```

-Wextra 提示: warning: missing initializer for field 'name' of 'struct Node'
// 警告: 没有初始化 'name' 成员

屏蔽某条警告: -Wno-<警告名>, 如: -Wno-missing-field-initializers



Cppcheck 静态代码分析

- ❖ Cppcheck 是一款优秀的静态代码分析工具专注于发现逻辑错误和未定义行为，尤其是在指针操作和内存管理方面

用法: `cppcheck --enable=all --inconclusive test.c`

- ∞ `--enable=all` 开启所有类型的检查（包括编码风格、性能建议等）
- ∞ `--inconclusive` 允许报告那些可能存在但不确定的问题

错误类型	GCC (-Wall -Wextra)	Cppcheck
语法错误	极强	弱（它假设代码能通过编译）
内存泄漏	基本无法检测	极强
逻辑顺序错误	较弱	强（如先用指针后判空）
缓冲区溢出	仅限简单情况	强（静态分析能力更深）

在实际开发中，通常先用 GCC 消除所有警告，再跑 Cppcheck 寻找深层 Bug



Cppcheck 静态代码分析（例）

- ❖ 例：内存泄漏，能识别出路径中未释放的内存，特别是提前返回的情况

```
void test_leak(int condition) {  
    char *p = (char *)malloc(100);  
    if (condition > 10) {  
        return; // 错误：在这里返回时，p 还没释放  
    }  
    free(p);  
}
```

Cppcheck 报告： [test.c:7]: (error)
Memory leak: p

- ❖ 例：空指针解引用，分析指针的逻辑流，找出为 NULL 仍被访问的情况

```
void null_pointer_check(int *p) {  
    *p = 10; // 1. 先解引用了  
    if (p == NULL) { // 2. 然后才检查是否为 NULL  
        return;  
    }  
}
```

Cppcheck 报告： [test.c:2]: (warning)
Possible null pointer dereference: p



Cppcheck 静态代码分析 (例续)

- ❖ 例：数组越界，对于静态数组或已知大小的缓冲区，能识别越界风险

```
void array_bounds() {  
    int arr[5];  
    for (int i = 0; i <= 5; i++) { // 错误：当 i=5 时越界  
        arr[i] = i;  
    }  
}
```

Cppcheck 报告： [test.c:4]: (error)
Array 'arr[5]' accessed at index 5,
which is out of bounds.

- ❖ 例：释放后使用，野指针的操作中最危险的行为之一，常导致程序崩溃

```
void use_after_free() {  
    char *p = malloc(10);  
    free(p);  
    p[0] = 'a'; // 错误：内存已释放，p 成了野指针  
}
```

Cppcheck 报告： [test.c:4]: (error)
Dereferencing 'p' after it is being
deallocated / released



CMake中集成Cppcheck方法

- ❖ CMake 3.10以上版本提供了一个内置变量，只要设置了它，每次编译文件时，CMake 都会自动调用 Cppcheck 扫描该文件，如果代码有逻辑漏洞，终端会直接跳出警告。只需在 CMakeLists.txt 中添加：

```
# 寻找系统中的 cppcheck 程序
find_program(CPPCHECK_EXE NAMES cppcheck)
if(CPPCHECK_EXE)
    message(STATUS "Cppcheck found: ${CPPCHECK_EXE}")
    # 设置 Cppcheck 的参数
    set(CMAKE_CXX_CPPCHECK
        ${CPPCHECK_EXE}
        "--enable=all inline-suppr"
        "--suppress=missingIncludeSystem"
    )
    # 如果是 C 项目，使用 CMAKE_C_CPPCHECK
    set(CMAKE_C_CPPCHECK ${CMAKE_CXX_CPPCHECK})
else()
    message(WARNING "Cppcheck not found! Static analysis skipped.")
endif()
```




内存调试技巧 (续)

❖ 动态分析工具：使用工具动态监视内存使用，排查内存问题

☞ Valgrind (Memcheck): 通过模拟 CPU 执行来监控每一次内存读写

❖ 检测目标：内存泄漏、非法读写、使用未初始化的值

❖ 使用方法：`valgrind --leak-check=full ./your_program`

❖ 注意：编译时要加上 `-g` 参数，这样才能报告代码出错的具体行号

☞ AddressSanitizer (ASan): 编译器内置，直接集成在可执行文件中

❖ 使用方法：GCC 4.8+, Clang 编译时加上 `-fsanitize=address -g`

❖ 优点：检测栈溢出和堆溢出，报错信息直接指明哪块内存被越界访问了

❖ 日志记录：记录每一块内存的分配记录，并对比释放记录

☞ 常用方法：自己写一个 `my_malloc` 和 `my_free` 的内存分配包装器

```
void* my_malloc(size_t size) {  
    printf("Allocated %zu bytes at %p [%s:%d]\n",  
           size, p, __FILE__, __LINE__);  
    return malloc(size);  
}
```



Valgrind 动态内存分析 (例)

- ❖ 例：非法读写，通常发生在数组越界或访问已经 free 掉的内存

```
int main() {  
    int *arr = malloc(10 * sizeof(int));  
    arr[10] = 5; //错误：索引从 0-9, arr[10] 越界写入  
    free(arr); int x = arr[0]; //错误：非法读取已经释放的内存  
    return 0;  
}
```

Valgrind 报告：Invalid write of size 4
(在 arr[10] 处), Invalid read of size 4
(在 free 后的读取处)

- ❖ 例：使用未初始化的值（可能是一个“随机”的内存垃圾值）

```
int main() {  
    int a; // 局部变量，未初始化  
    if (a == 0) // 错误：a 的值是不确定的  
        printf("A is zero\n");  
    return 0;  
}
```

Valgrind 报告：Conditional jump or
move depends on uninitialised
value(s) (if 跳转取决于一个不确定变量)



Valgrind 动态内存分析 (例续)

- ❖ 例：内存泄漏，借了内存 (malloc)，用完不还 (free)

```
void leak_demo() {  
    int *p = malloc(100 * sizeof(int));  
    // 应该在这里写 free(p);  
    return; // 函数结束，p 消失了，但堆内存还在，彻底“失联”  
}  
  
int main() {  
    leak_demo(); return 0;  
}
```

Valgrind 报告总结：

- Definitely lost: 肯定丢了（必须修复）。
- Indirectly lost: 间接丢了（比如结构体内部内存）。
- Still reachable: 程序退出时还没释放，但指针还在

- ❖ 例：非法释放，包括重复释放或释放了并不是由 malloc 分配的指针

```
int main() {  
    int *p = malloc(10 * sizeof(int)); free(p);  
    free(p); // 错误：Double Free!  
    int x = 10; int *p2 = &x;  
    free(p2); // 错误：释放了栈上的变量  
    return 0;  
}
```

Valgrind 报告：Invalid free() / delete / delete[] / realloc()（会报告该地址之前在哪行已经被释放过了）



ASan 动态内存分析 (例)

- ❖ 例：堆内存越界，访问了 malloc 分配范围之外的内存

```
int main() {  
    char *heap_buf = (char *)malloc(10 * sizeof(char));  
    // 错误：试图写入第 11 个字节（索引 10）  
    heap_buf[10] = 'x';  
  
    free(heap_buf);  
    return 0;  
}
```

ASan 报告： heap-buffer-overflow
(提供影子内存的状态，提供报错位置前后的内存分布)

- ❖ 例：栈内存越界，在检测栈溢出方面是 ASan 的强项

```
int main() {  
    int stack_array[5] = {1, 2, 3, 4, 5};  
    // 错误：访问了栈上定义的数组之外的内容  
    int val = stack_array[5];  
  
    printf("Value: %d\n", val);  
    return 0;  
}
```

ASan 报告： stack-buffer-overflow
(在栈变量周围注入红区，一旦踩到红区，警报立刻拉响)



ASan 动态内存分析 (例续)

- ❖ 例：释放后使用，即所谓的“悬空指针”问题

```
int main() {  
    int *p = malloc(sizeof(int)); *p = 42;  
    free(p);  
    // 错误：内存已经还给系统了，不能再碰  
    printf("Read after free: %d\n", *p);  
    return 0;  
}
```

ASan 报告： heap-use-after-free (这块内存存在代码的哪一行被分配，又是在哪一行被释放的)

- ❖ 例：内存泄漏，在程序退出时会自动检查

```
void create_leak() {  
    int *leak = malloc(64);  
    // 忘记 free(leak);  
}
```

ASan 报告： detected memory leaks (直接列出泄露对象的起始地址和大小)

```
int main() { create_leak(); return 0; }
```



内存调试技巧总结

❖ 内存调试技巧优缺点

技巧	适用场景	优点	缺点
GDB	崩溃定位	实时交互，查看状态	难发现静默的内存泄漏
Valgrind	深度排查	最彻底，查得最准	运行速度极慢 (慢 10-50 倍)
ASan	日常开发	速度快，集成度高	无法检测某些复杂逻辑泄漏
日志记录	长期运行测试	还原分配轨迹	增加代码量和性能开销

❖ 分阶段调试与优化关注点

阶段	关注点	核心工具/方法
开发阶段	代码健壮性	静态代码分析 (Cppcheck), 安全函数
测试阶段	捕捉 Bug	Valgrind, ASan, GDB
发布阶段	运行速度	缓存优化, 内存对齐, -O2/-O3 编译优化



内存优化概述

- ❖ 内存优化不仅仅是“节省空间”，更核心的目的是提高程序的运行效率（速度）和增强系统的稳定性
- ❖ 内存优化通常从三个维度展开：**数据结构优化**（空间）、**访问模式优化**（速度）和**管理策略优化**（效率）
- ❖ 内存优化不是越极端越好，首先保证逻辑正确，找到性能瓶颈，有针对性的进行优化，过度优化会降低代码的可读性
- ❖ 常见的内存浪费陷阱：优化的最高境界是程序没有优化空间
 - ∞ 过度分配：习惯性地申请 [1024] 长度的内存，其实只用到 10 字节
 - ∞ 重复存储：多个结构体存储了相同的长字符串，可考虑使用常量池
 - ∞ 内存泄漏：虽属于 Bug，但长期运行的泄漏是最大的内存优化敌手



数据结构优化：结构体对齐与重排

- ❖ 在不同的系统里，即使各基本类型所占内存大小一致，下面两个结构体的 sizeof 值也很可能不一样，为什么？

```
struct _a{  
    char c1;  
    long i;  
    char c2;  
} a;
```

sizeof(a)=12

```
struct _b{  
    char c1;  
    char c2;  
    long i;  
} b;
```

sizeof(b)=8

假设： sizeof(char)=1, sizeof(long)=4

- ❖ 在分配结构体空间的时候，存在地址对齐的要求



数据结构优化：结构体对齐与重排

❖ 为什么要对齐？

- ❧ 某些平台对特定类型的数据只能从特定地址开始存取
- ❧ 提高内存访问效率，便于CPU的缓存调度

❖ 结构体字节对齐的细节和[具体编译器实现相关](#)（不同的系统可能存在差异），一般而言满足三个准则：

1. 结构体变量的首地址能够被实际对齐单位所整除；
2. 每个成员相对于结构体首地址的偏移量 (offset) 都是成员大小或实际对齐单位的整数倍，否则会在成员之间加上填充字节；
3. 结构体的整体大小必须为实际对齐单位的整数倍，否则会在最末一个成员之后加上填充字节 (trailing padding)。

❖ 实际对齐单位与最宽基本类型成员的大小、预编译指令等有关



数据结构优化：结构体对齐与重排

❖ 编译器为了对齐地址，会在结构体成员间插入填充字节

```
struct _a{  
    char c1;  
    long i;  
    char c2;  
} a;
```

0	c1
1	-
2	-
3	-
4	i
5	
6	
7	
8	c2
9	-
10	-
11	-

sizeof(a)=12

```
struct _b{  
    char c1;  
    char c2;  
    long i;  
} b;
```

0	c1
1	c2
2	-
3	-
4	i
5	
6	
7	

sizeof(b)=8

假设对齐要求为 char: 1, long: 4

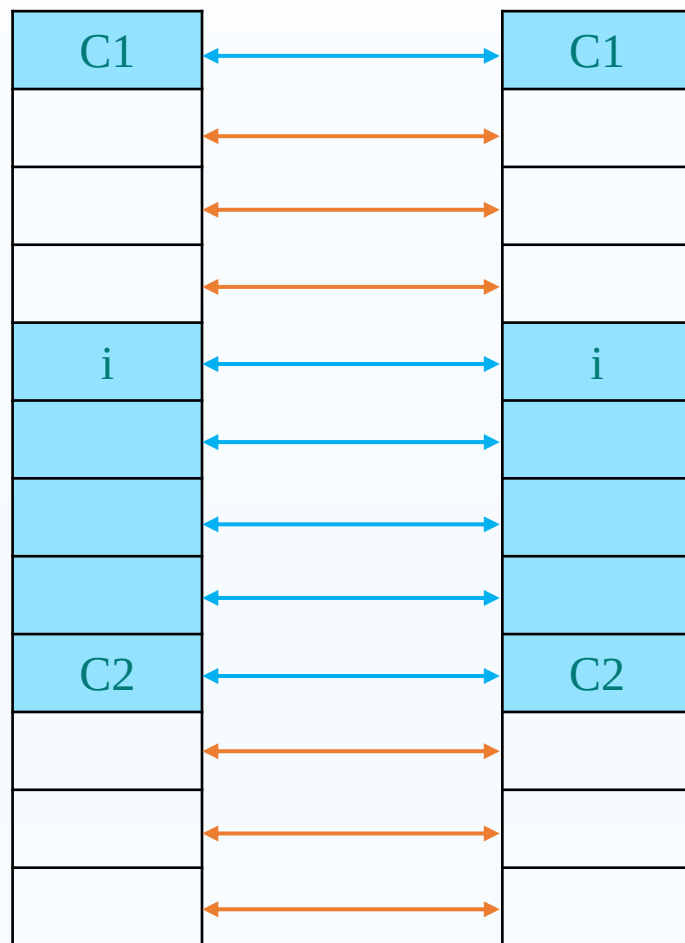


数据结构优化：结构体对齐与重排

❖ 结构体中“可能的空洞”的存在，决定了结构体变量间不能进行相等 == 或不等 != 的运算

- ∞ 逐个比较成员是极没有效率的
- ∞ 比较全部字节是一个较好的选择，但是空洞使得比较无法进行

```
struct _a{  
    char c1;  
    long i;  
    char c2;  
} a;
```





数据结构优化：结构体对齐与重排

- ❖ 对于结构体数组，因地址对齐而导致的内存浪费，值得重视

```
struct Student_info1{  
    char name[10];  
    long num;  
    char gender;  
    int age;  
} stu1[10000];
```

```
struct Student_info2{  
    char name[10];  
    char gender;  
    long num;  
    int age;  
} stu2[10000];
```

```
sizeof(char)=1  
sizeof(int )=4  
sizeof(long)=4  
char: 1  
int : 4  
long: 4
```

∞ sizeof(struct Student_info1)=24; sizeof(stu1)=240KB

∞ sizeof(struct Student_info2)=20; sizeof(stu2)=200KB



数据结构优化：结构体位域

❖ 位段是以位为单位定义长度的结构体类型成员

❖ 定义形式：

```
struct 位域结构名 {  
    位段成员列表;  
};
```

位段成员的说明形式为：

类型说明符 位段名:位段长度;

```
struct packed_data {  
    unsigned int a:2;  
    unsigned int b:6;  
    unsigned int c:4;  
    unsigned int d:4;  
    unsigned int :0;  
    int i; 普通结构体成员  
};
```

位段成员

普通结构体成员

❖ 注意：

∞ 一个位段分配在同一个存储单元之中，不能跨单元

❖ 可以用 unsigned int:0; 表示从下一个存储单元开始存放

❖ 可以用未命名的位段来填充或调整位置，例如 unsigned int:3;



数据结构优化：结构体位域（例）

```
struct Date {  
    unsigned int d;  
    unsigned int m;  
    unsigned int y;  
};  
// 输出: 12 bytes, 每个存储单元 4 bytes  
printf("%lu bytes\n", sizeof(struct Date));
```

```
struct Date {  
    unsigned int d : 5; // 0~31天, 5 bits 就够了  
    unsigned int m : 4; // 0~12月, 4 bits 就够了  
    unsigned int y;  
};  
// 输出: 8 bytes, 共两个存储单元  
printf("%lu bytes\n", sizeof(struct Date));
```



数据结构优化：结构体位域（例续）

```
struct Date {  
    unsigned int d : 5;    // 0~31天, 5 bits 就够了  
    unsigned int m : 4;    // 0~12月, 4 bits 就够了  
    unsigned int y;  
};  
// 输出: 8 bytes, 共两个存储单元  
printf("%lu bytes\n", sizeof(struct Date));
```

```
struct date {  
    unsigned int d : 5;  
    unsigned int : 0;    // 从下一个存储单元开始  
    unsigned int m : 4;  
    unsigned int y;  
};  
// 输出: 12 bytes, 共三个存储单元  
printf("%lu bytes\n", sizeof(struct Date));
```



数据结构优化：结构体位域

❖ 使用中应注意成员所占的位及其由长度限定的存取值域

☞ 溢出了会怎样？与编译器的实现相关。

❖ 不能对位段应用取地址运算

☞ 地址的最小运算单位是字节 (byte)

❖ 不能用指针指向位段

```
struct S { unsigned int x : 5; }; S s;
```

```
printf("%p", &s.x); // 错误
```

❖ 位段数组不被允许

```
struct S { unsigned int x[10] : 5; };
```

```
// 错误
```

```
#include <stdio.h>
struct Test {
    unsigned int x : 2;
    unsigned int y : 2;
    unsigned int z : 2;
};
void main() {
    struct Test t;
    t.x = 5;
    printf("%d %d",
            t.x, t.y);
}
// 可能的输出: 1 0
// 溢出的位被丢弃
// 未影响另一成员
```




数据结构优化：联合体类型

- ❖ 有时为优化内存使用，需要将不同类型变量存储在同一段存储单元中，可以定义联合体类型，也叫共用体类型

- ❧ 任一时刻，只使用其中一个变量

- ❧ 本质是以不同方式解读该段内存的数据

- ❖ 联合体的定义与用法类似结构，定义格式

```
union 共用体名 {  
    类型1 成员名1;  
    类型2 成员名2;  
    ... ..  
    类型n 成员名n;  
};
```

```
union ifcd {  
    int    i;  
    float  f;  
    char   c;  
    double d;  
};
```



数据结构优化：联合体类型

❖ 联合体的变量定义

∞ 先声明联合类类型，后声明变量

```
union ifcd x1,x2[5],*px;
```

∞ 或直接在定义联合体类型时声明变量

❖ 联合体变量的引用

∞ 与结构体类型相同，使用 . 及 -> 访问成员

∞ 本质上是以不同的类型来读取同一段存储单元

❖ 联合体与结构体的区别

∞ 结构体成员各自占用分配给自己的存储单元，联合体中的成员占用同一个存储单元

∞ 联合体变量初始化时，只能也只需对第一个成员赋值



数据结构优化：联合体类型（例一）

- ❖ 利用联合体类型，
可以构造混合的数据结构

```
typedef union {  
    int i;  
    double d;  
} Number;  
Number num_array[1000];
```

- ❖ 可以混合结构体类型和联合体类型，
来为联合添加“标记字段”，
以记录联合体包含是什么数据

```
typedef struct {  
    char tag;  
    union{  
        int i;  
        double d;  
    } u;  
} Number;
```



数据结构优化：联合体类型（例二）

- ❖ 使用联合体类型可以提供数据的多个视角

```
union int_date {  
    long i;    //假定long是8字节  
    struct date_time{  
        int hour;    //假定int是4字节  
        int minute;  
    };  
};
```

- ❖ 可以方便地将一个长整数与文件日期相互转换



数据结构优化总结

- ❖ 结构体对齐与重排：为了对齐地址，会在结构体成员间插入填充字节

☞ 原则性技巧：将占用空间大的变量排在前面，占用小的排在后面

```
// 优化前：占 12 字节 (char后补3字节，第二个char后补3字节)
struct Bad { char a; int b; char c; };
// 优化后：占 8 字节 (两个char紧挨着，共占2字节，补2字节对齐int)
struct Good { int b; char a; char c; };
```

- ❖ 结构体位域：如果只需要存储小范围数值，没必要用整个数据类型内存

☞ 优化技巧：在结构体中使用：宽度 定义变量

```
struct Flags {
    unsigned int is_active : 1;    // 只占 1 bit
    unsigned int priority  : 3;    // 占 3 bits (0-7)
};
```

- ❖ 联合体：当多个变量不会同时使用时，使用联合体让其共享同一块内存



访问模式优化：内存块的赋值与拷贝

❖ 批量赋值函数

```
#include <string.h>
```

```
void *memset ( void *ptr, int value, size_t num );
```

- ☞ 为指针 ptr 所指内存空间的前 num 个字节，赋值为 value
- ☞ value 参数的类型为 int 型，执行时系统解析为 unsigned char (0~512)
- ☞ 经常用来为数组清零，如：int a[10]; memset(a, 0, sizeof(a));

❖ 整块拷贝函数

```
#include <string.h>
```

```
void *memcpy ( void *dest, const void *src, size_t num );
```

- ☞ 将指针 src 所指内存的前 num 个字节拷贝到指针 dest 所指的内存
- ☞ 拷贝是以字节为单位进行的，与数据类型无关，dest 所指的空间（已分配）需要足够大，避免内存溢出
- ☞ dest和src的空间不能重叠，如果有重叠需要使用 memmove 函数来完成
- ☞ 经常用于数组的拷贝，如int a[10], b[10]; memcpy(b, a, sizeof(b));



访问模式优化：内存块的赋值与拷贝

- ❖ 在数组/结构体的操作中，虽然循环最直观，但memset和memcpy效率更高：循环通常是“逐元素”操作的，而后者是“按块”操作的

```
#include <string.h>
int main() {
    int src[100000] = {...};
    int dest[10000];

    for (int i = 0; i < 100000; ++i)
        dest[i] = 0;                                // 效率低
    memset(dest, 0, sizeof(dest));                    // 效率高

    for (int i = 0; i < 100000; ++i)
        dest[i] = src[i];                             // 效率低
    memcpy(dest, src, sizeof(src));                    // 效率高

    return 0;
}
```



访问模式优化：内存块的赋值与拷贝

❖ 循环与memset / memcpy对比总结

特性	循环	memset / memcpy
执行速度	较慢（逐元素）	极快（寄存器级/SIMD 优化）
代码量	较多（3-4 行）	极简（1 行）
灵活性	高（可以处理非连续逻辑）	低（只能处理连续内存块）
类型安全	较好（基于元素类型）	较低（基于字节，需注意类型宽度）
场景建议	逻辑复杂、非 0 初始化	块拷贝、结构体清零

❖ memset：按字节执行，最适合将内存置零（0）的场景

❧ **错误示范**：使用 `memset(a, 1, sizeof(a))` 试图把 `int` 数组全设为 1。
由于是按字节填充，最终每个 `int` 会变成 `0x01010101`（即 16843009）

❖ memcpy：第三个参数是总字节数，而不是元素个数

❧ **内存重叠**：memcpy 不保证在源地址和目标地址重叠时能正确工作，可能会导致数据损坏；如果发生重叠，必须使用更安全的 `memmove`



访问模式优化：结构体赋值优化

- ❖ 可将一个有值的结构变量整体赋给另一个同类型的结构变量
`struct Student d1={1400101,"Liu",'M',19,92}, d2;`
`d2 = d1;`
- ❖ 当结构体作为形参/实参时，采用的是值传递，整个结构体的数据都会被复制，当结构体较大时效率较低
 - ☞ 使用const传递结构体指针：`void f (const struct Student *p1);`
- ❖ 即使结构体中包含数组，也会被整体赋值
 - ☞ 数组不能整体赋值，封装在结构体中就可以，其采用DMA高速通道，可能比自己写循环复制的效率更高（效果类似 `memcpy`）
 - ☞ 利用结构体封装，也可以将整个数组作为函数实参进行传递



访问模式优化：提高CPU缓存命中率

- ❖ CPU 的运算速度极快，而内存相对极慢，为此设计了三级缓存（L1, L2, L3），从内存取数据时，每次加载一个缓存行（通常是 64 字节）

∞ **空间局部性**：数组在内存中是连续存储的。按行访问二维数组比按列访问快得多，因为按行访问能命中缓存行

```
// 优秀：缓存友好（行优先）
for (int i = 0; i < ROWS; i++)
{
    for (int j = 0; j < COLS; j++)
    {
        sum += matrix[i][j];
    }
}
```

```
// 糟糕：缓存杀手（列优先）
for (int j = 0; j < COLS; j++)
{
    for (int i = 0; i < ROWS; i++)
    {
        sum += matrix[i][j];
    }
}
```

∞ **减少指针跳转**：链表在内存中是分散的，每次追踪 next 指针都可能导致一次缓存缺失，如果数据量固定，优先使用连续数组代替链表



分配策略优化

- ❖ **优先使用栈内存**：栈分配只需要移动一次栈指针，而堆分配（`malloc`）涉及复杂的链表搜索和系统调用
 - ⌘ 优化技巧：小块、生命周期短的数据尽量在栈上分配（函数内）
- ❖ **考虑使用内存池**：预先申请大块内存自行管理，避免内存碎片，将多次 `malloc` 合并为一次，极大提升分配速度
- ❖ **延迟分配与重用**
 - ⌘ 延迟分配：直到真正需要使用内存时才调用 `malloc`
 - ⌘ 对象重用：对于频繁使用的结构体，释放后不要直接 `free`，而是放入一个“空闲链表”中，下次直接复用



Linux系统中程序内存占用检查

❖ 实时交互式查看

- ❧ `top` 命令: 系统自带, 查看 RES (物理内存) 和 VIRT (虚拟内存) 列, 按下 M 键可以按内存占用排序
- ❧ `htop` 命令: `top` 的增强版 (界面更友好, 需要安装), 不仅有进度条显示, 还能直观看到进程的内存占比

❖ 静态快照查看

- ❧ `ps aux` 命令: RSS 列是进程实际占用的物理内存 (单位: KB), %MEM 列是占用物理内存的百分比, 常用组合 `ps aux --sort=-rss | head`
- ❧ `pmap -x <PID>` 命令: 可以展示该进程每一块内存地址空间 (代码段、堆、栈、共享库) 分别占用了多少内存, 最后一行 total RSS 是最关键的参考值

❖ 内存调试工具

- ❧ Valgrind (Massif): 运行 `valgrind --tool=massif ./your_program`, 它会生成一个文件, 记录程序运行全过程的内存曲线



内存优化技巧小结

- ❖ 在进行内存优化时，可以通过简单的公式估算内存利用率：

$$\text{利用率} = \frac{\sum \text{实际有效载荷数据}}{\text{程序总占用内存}} \times 100\%$$

- ❖ 优化的目标就是让这个百分比尽可能接近 100%，同时保持 CPU 的缓存命中率在高位
 - ∞ 数据结构优化（空间）：结构体对齐与重排、结构体位域、联合体
 - ∞ 访问模式优化（速度）：内存块拷贝与赋值、尽可能CPU缓存友好
 - ∞ 管理策略优化（效率）：减少堆分配次数（`malloc`）、使用内存池



谢谢!



中国科学技术大学

University of Science and Technology of China